TRIBHUVAN UNIVERSITY
Institute of Science and Technology
Department of Computer Science and Information Technology
Patan Multiple Campus, Lalitpur

FINAL YEAR PROJECT REPORT

On

"NYAYAK: DIGITAL LEGAL NAVIGATION SYSTEM DESIGNED FOR NEPAL"

In partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Information Technology
(B.Sc. CSIT)

Submitted to:
Department of Computer Science and Information Technology
Patan Multiple Campus, Lalitpur

Submitted By:
Samir Paudel (Roll No: 79010103)
Prakashman Singh Thakuri (Roll No: 79012186)

Supervisor: [Deo Narayan Yadav, Department of CSIT]

SUPERVISOR RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **Samir Paudel** and **Prakashman Singh Thakuri**, entitled "NYAYAK: DIGITAL LEGAL NAVIGATION SYSTEM DESIGNED FOR NEPAL," in partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Information Technology, be processed for evaluation.

. . . . . .

Mr. Deo Narayan Yadav

Project Supervisor

Department of Computer Science and Information Technology

Patan Multiple Campus

APPROVAL LETTER

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Tribhuvan University**, **Institute of Science and Technology**, and the **Department of Computer Science and Information Technology, Patan Multiple Campus** for providing us the opportunity to carry out this project.

We are especially thankful to our project supervisor, **Mr. Deo Narayan Yadav**, for his valuable guidance, encouragement, and continuous support throughout the completion of this project.

We also extend our thanks to all faculty members, staff, friends, and well-wishers whose support and suggestions helped us complete this work successfully.

Finally, we are deeply grateful to our families for their constant motivation, patience, and support during the entire project period.

Samir Paudel

Roll No: 79010103

Prakashman Singh Thakuri

Roll No: 79012186

ABSTRACT

Nyayak is a full-stack digital legal navigation platform designed for Nepal's legal corpus and court infrastructure. The system addresses three persistent barriers to justice: limited public understanding of rights under the Constitution of Nepal [1] and the Civil Code 2017 [2], weak digital visibility for qualified lawyers outside major urban centres, and the lack of structured court information across Nepal's 77 districts. It integrates an AI legal assistant, a lawyer marketplace, a community forum, and a court navigation module within a single web platform built on Next.js, Convex, and FastAPI. The legal assistant uses a hybrid RAG pipeline that combines BM25 retrieval with sentence-transformer semantic search, a design supported by prior work on RAG and hybrid retrieval [3–5].

The lawyer marketplace applies content-based recommendation to rank suitable legal professionals [6, 7], while the forum uses a PageRank-inspired influence model [8] and the court navigator uses shortest-path routing over a weighted district-level graph [9]. Evaluation on 50 legal queries achieved an 86% source-citation relevance rate, average latency of 3.2 seconds, and full success across eight system test cases. Deployed on free-tier infrastructure and publicly accessible at `https://nyayak.vercel.app`, Nyayak demonstrates that modern retrieval, recommendation, and graph-based methods can support an accessible and practical digital legal guidance platform for Nepal.

Keywords: Retrieval-Augmented Generation, Legal AI, Nepal Constitution, Hybrid Search, BM25, Semantic Similarity, PageRank, Dijkstra's Algorithm, Content-Based Filtering, Full-Stack Web Application, Next.js, FastAPI, Convex, Access to Justice

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BM25	Best Match 25 (Probabilistic Ranking Function)
CORS	Cross-Origin Resource Sharing
CSIT	Computer Science and Information Technology
DFS	Depth-First Search
ER	Entity Relationship
FastAPI	Fast Application Programming Interface (Python framework)
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
IDF	Inverse Document Frequency
JWT	JSON Web Token
LLM	Large Language Model
MAU	Monthly Active Users
NLP	Natural Language Processing
NPR	Nepalese Rupee
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SHA	Secure Hash Algorithm
SSR	Server-Side Rendering
TF	Term Frequency
TF-IDF	Term Frequency-Inverse Document Frequency
TU	Tribhuvan University
UI	User Interface
UX	User Experience

List of Figures

List of Tables

1. Introduction

1.1. Introduction

Access to legal services and information remains one of the most significant and least-addressed barriers faced by citizens across developing nations. Nepal is no exception. The Nepali legal system - governed by a comprehensive Constitution ratified in 2015 (2072 B.S.) [1], supplemented by the Civil Code 2017 [2], the Labour Act 2074, the Land Act 2021, and dozens of provincial statutes - is complex, multilingual, and largely inaccessible to ordinary citizens without professional legal counsel. With more than 150,000 cases pending in courts across the country, the gap between those who can effectively navigate the justice system and those who cannot continues to widen.

The access-to-justice deficit in South Asia is well-documented in academic literature. Information asymmetry - not merely cost - has been identified as a major barrier to legal participation, as citizens who lack basic awareness of their legal rights are structurally excluded from the justice system regardless of formal legal guarantees. This challenge is directly applicable to Nepal, where the Constitution guarantees equality before the law and the right to legal representation, yet no reliable digital resource exists to translate those guarantees into accessible, plain-language guidance. Recent work in legal NLP also shows growing interest in tools that help users search, summarise, and interpret legal documents [10].

Nyayak is a digital legal navigation platform designed to bridge this access gap. The name derives from the Nepali word Nyayak, meaning “legal advocate” or “one who upholds justice,” reflecting the system’s core mission: delivering justice-enabling technology to every Nepali citizen regardless of socioeconomic status, geographic location, or level of legal literacy. The platform operates as a three-layer solution combining an AI-powered legal assistant grounded in Nepal’s constitutional and civil corpus, a verified lawyer marketplace with intelligent content-based matching, and a community-driven knowledge ecosystem supported by a graph-ranked forum.

The platform is built on a modern full-stack architecture: a Next.js 14 frontend with Convex for real-time reactive data, Clerk for identity management, and a Python FastAPI backend powered by LangChain, ChromaDB, and large language models via the Groq API. Next.js provides server-side rendering capabilities that improve page load performance and search engine accessibility compared to client-side-only approaches [11]. The AI assistant employs a Hybrid Retrieval-Augmented Generation (RAG) pipeline combining BM25 probabilistic ranking and dense cosine semantic similarity - a design choice informed by empirical

evidence that hybrid retrieval systems consistently outperform single-method approaches on complex knowledge-intensive tasks [5, 12]. Large language models without grounding in external knowledge sources are known to produce legally inaccurate outputs at alarming rates, with studies documenting hallucination rates between 58% and 88% on specific legal question tasks [13, 14]; RAG architectures are the primary mechanism for mitigating this risk in the legal domain [15].

Nepal's internet penetration rate reached approximately 58% in 2023, validating the feasibility of a web-based access mechanism for the target demographic. Nyayak is deployed entirely on free-tier cloud infrastructure and is publicly accessible at `https://nyayak.vercel.app`, ensuring no financial barrier to adoption.

1.2. Problem Statement

This subsection identifies the practical access-to-justice problems that motivated Nyayak. The issues are grouped around citizens' difficulty obtaining reliable legal guidance, lawyers' limited digital visibility, and the absence of accessible court and jurisdiction information.

i. Information Asymmetry for Citizens

Legal consultations in Nepal are prohibitively expensive for a significant portion of the population, and even locating a lawyer with the correct specialisation is difficult without personal referrals. Citizens facing land disputes, domestic violence, contract breaches, or employment violations frequently lack even basic awareness of their constitutional rights. No publicly accessible digital resource provides authoritative, cited answers to legal questions grounded in Nepali law. Large language models deployed without domain-specific retrieval grounding have been shown to produce plausible-sounding but factually incorrect legal information, a particularly dangerous failure mode in high-stakes legal contexts [13]. The integration of RAG with verified legal corpora is the established solution to this hallucination problem in legal AI systems [14].

ii. Market Inefficiency for Legal Professionals

Practising lawyers and advocates - particularly those outside Kathmandu - have no centralised platform to showcase credentials, publish specialisation areas, or connect with prospective clients whose cases match their expertise. This results in systematic underutilisation of legal talent in the provinces while demand remains unmet in underserved communities. Content-based recommendation systems represent the canonical solution for such structured profile-matching problems, as they do not require historical interaction data and can produce

interpretable, explainable recommendations from structured item profiles alone [7].

iii. Absence of Court Infrastructure Information

Court locations, jurisdictions, operating hours, and case-filing procedures are poorly documented in any accessible digital form. Nepal has 77 districts, each with its own court hierarchy, and the relationships between districts and jurisdictions are complex and inconsistently documented online. Citizens navigate these institutions largely through informal channels, adding unnecessary cost and delay to already overburdened proceedings. Graph-theoretic shortest-path algorithms, such as Dijkstra's algorithm, provide an efficient computational solution for geographic routing under weighted constraint networks, with well-established time complexity guarantees that make them practical for real-time navigation applications [9].

Existing global platforms such as LegalZoom (USA) and Kanoon.co (India) address similar challenges but are focused on their respective national jurisdictions. No comparable platform exists for Nepal. The Nepal Bar Council's official digital presence is limited to registration and directory functions, with no advisory or matching capabilities.

1.3. Objectives

The primary objective of Nyayak is to develop a fully functional, accessible, and AI-augmented legal navigation platform tailored to the laws and legal infrastructure of Nepal. The specific objectives are:

- **AI-Powered Legal Assistant:** To develop an AI-powered Legal Assistant using a hybrid Retrieval-Augmented Generation (RAG) pipeline combining BM25 keyword search and semantic cosine similarity to retrieve and synthesise authoritative answers from the Constitution of Nepal [1] and the Civil Code 2017 [2], providing citizens with instant, citation-backed legal guidance in plain language [3].
- **Verified Lawyer Marketplace and Case Repository:** To design and implement a Verified Lawyer Marketplace and Case Repository with a content-based recommendation engine that intelligently matches citizen queries and relevant case needs to qualified legal professionals based on specialisation, profile relevance, and availability - enabling efficient case discovery and better lawyer-case visibility [7].
- **Community Legal Forum and Court Navigation:** To build a Community Legal Forum and Court Navigation Module that empowers citizens through a graph-ranked discussion forum and locates the nearest jurisdictionally appropriate court using Dijkstra's shortest-path algorithm across Nepal's 77-district court network [9, 16].

- **Real-Time Chatting System (Future Integration *):** To incorporate a real-time chatting system that enables secure and direct communication between citizens and lawyers for consultation, follow-up, and case-related support.

1.4. Scope and Limitation

This subsection defines the functions, users, legal resources, and geographic coverage included in the current Nyayak implementation. It also identifies the technical and operational boundaries that remain outside the present project version.

i. Scope

The AI legal assistant is scoped to the Constitution of Nepal (2072 B.S.) and Civil Code 2017 as the primary retrieval corpus.

The lawyer marketplace covers all primary legal specialisations including property, criminal, family, labour, corporate, and constitutional law.

The court navigation module covers all 77 districts of Nepal, providing district and high court routing at minimum.

The community forum supports nested threaded discussion, contributor reputation scoring, and real-time interaction.

The platform is accessible via modern web browsers on desktop and mobile without installation.

ii. Limitations

The current implementation does not support full Nepali Devanagari script queries; queries are accepted in English and Nepali-English transliteration.

Lawyer verification is performed through a manual administrative review process and is not yet integrated with the Nepal Bar Council's official registry API.

The RAG pipeline is evaluated against the constitutional corpus only; statute-specific accuracy depends on future corpus expansion to provincial legislation and procedural statutes.

The system is deployed on free-tier cloud infrastructure, which imposes cold-start latency constraints not present in production environments with persistent compute.

1.5. Development Methodology

Nyayak employs an incremental, module-by-module development methodology. Each component is built, tested, and integrated independently before the next begins. This approach maintains a functional version of the application at all times, ensures regressions are caught early, and aligns with the fourteen-week project timeline of the seventh semester.

Development is organised into four phases: (1) Requirements and Architecture (Weeks 1–3), establishing the technology stack, database schema, and legal corpus; (2) Core AI and Recommendation Backend (Weeks 3–7), building the RAG pipeline and lawyer matching engine; (3) Application Module Development (Weeks 5–11), implementing the marketplace, forum, and court navigator; and (4) Testing and Documentation (Weeks 11–14), covering end-to-end testing, result analysis, and final report preparation.

1.6. Report Organization

This report is organised into six chapters. Chapter 1 provides the project introduction, problem statement, objectives, scope, and development methodology. Chapter 2 presents the background study and literature review covering the fundamental theories, concepts, and related academic work that underpin the project. Chapter 3 details the system analysis including requirements analysis, feasibility study, and object-oriented modelling. Chapter 4 covers the system design including database design, the component diagram, and algorithm specifications. Chapter 5 describes the implementation details, tools used, testing methodology, test results, and result analysis. Chapter 6 presents the conclusion and future recommendations.

2. Background Study and Literature Review

2.1. Background Study

This section provides an overview of the fundamental theories, algorithms, frameworks, and terminologies that underpin the design and implementation of Nyayak.

2.1.1. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG), introduced by Lewis et al., grounds large language model responses in passages retrieved from an external document corpus [3]. A retriever ranks and returns the most relevant passages for a query, and a generator produces the answer using both the query and retrieved evidence. This reduces dependence on potentially outdated or incomplete knowledge stored only in model parameters.

RAG improves factual accuracy on knowledge-intensive tasks and allows the external knowledge corpus to be updated without retraining the language model [3]. This is particularly useful in legal applications because statutes and legal guidance may change while generated answers must remain traceable to current source documents.

In the context of Nyayak, RAG is the mechanism by which every AI-generated legal response is grounded to specific articles of the Constitution of Nepal [1] and the Civil Code 2017 [2]. Without RAG, an LLM responding to questions about Nepali law would produce plausible-sounding but potentially factually incorrect responses - a phenomenon known as "hallucination" - with rates documented between 58% and 88% for legal question-answering tasks across commercial models [13]. With RAG, the system can cite the exact constitutional article or civil code clause that supports each statement in the response, a property shown to be critical for responsible deployment of AI in legal contexts [15].

2.1.2. BM25 Probabilistic Ranking

BM25 (Best Match 25) is a probabilistic document ranking function derived from the Binary Independence Model [17] that scores a document d against a query Q by computing the weighted sum of per-term scores, where each term score accounts for term frequency (TF) within the document, inverse document frequency (IDF) across the corpus, and document length normalisation relative to the average corpus document length [18]. The standard tuning parameters $k_1 = 1.5$ and $b = 0.75$ control term frequency saturation and length normalisation respectively. The formula for BM25 scoring is expressed as:

$$\text{Score}_{BM25}(d,Q) = \sum_{t \in q} \text{idf}(t) \times \frac{\text{tf}(t,d) \times (k_1 + 1)}{\text{tf}(t,d) + k_1 \times \left(1 - b + b \times \frac{|d|}{\text{avgdl}}\right)}$$

BM25 remains a strong competitive baseline for legal document retrieval tasks. In the Competition on Legal Information Extraction/Entailment (COLIEE 2021), a vanilla BM25 system achieved a second-place ranking among 25 participating teams from 14 countries,

demonstrating that exact keyword matching remains highly effective for statutory text retrieval where precise legal terms, article numbers, and defined concepts appear verbatim in documents [18]. A hybrid approach combining BM25 with dense transformer-based retrieval further improves performance on multi-step legal reasoning tasks by capturing semantic paraphrases that BM25 alone misses [19]. In Nyayak's hybrid pipeline, BM25 contributes 70% of the final retrieval score, prioritising keyword precision critical for statutory citation accuracy.

2.1.3. Semantic Similarity with Sentence Transformers

Dense vector semantic search uses neural sentence embedding models to represent both queries and documents as high-dimensional vectors in a shared semantic space, with cosine similarity between these vectors capturing semantic relatedness even when the exact query words do not appear in the document. Sentence-BERT introduced Sentence-BERT (SBERT), which produces sentence-level embeddings optimised for semantic textual similarity tasks by fine-tuning BERT with a siamese network structure on natural language inference datasets . A key practical advantage of SBERT over standard BERT for similarity search is computational efficiency: finding the most similar pair in a collection of 10,000 sentences requires approximately 65 hours with BERT but only about 5 seconds with SBERT, while maintaining comparable accuracy - a reduction that makes real-time semantic retrieval feasible in production systems. Systematic evaluation of dense sentence-transformer encoders against BM25 on retrieval benchmarks shows that dense retrievers substantially outperform BM25 in ranking quality (nDCG@10 = 0.916 vs. 0.799), particularly for semantically paraphrased queries [5].

In Nyayak, semantic similarity handles cases where a citizen phrases a legal question in non-technical language that does not directly mirror statutory vocabulary. For example, a query about a landlord refusing to return a deposit may not directly mirror the legal term "bayana" but should still retrieve relevant provisions from the Civil Code. The semantic component contributes 30% of the final retrieval score, complementing BM25's keyword precision with semantic coverage. Empirical studies on hybrid retrieval confirm that combining sparse and dense retrievers through fusion mechanisms achieves the highest recall and lowest omission compared to either retrieval mode alone [20].

2.1.4. Content-Based Filtering for Recommendation

Content-based filtering is a recommendation approach that matches item profiles to user preference profiles based on the intrinsic attributes of items. Adomavičius and Tuzhilin

(2005) provide a comprehensive survey of the field, categorising content-based approaches as one of the three main paradigms - content-based, collaborative, and hybrid - and identifying their key advantage as independence from interaction history: a new item with a well-defined profile can be recommended immediately without requiring any engagement data [7]. This cold-start advantage is particularly important for professional marketplace applications such as Nyayak's lawyer directory, where new lawyers joining the platform must be immediately discoverable without accumulating historical consultation data. Knowledge-based recommendation systems, closely related to content-based approaches, reason explicitly about item attributes and user requirements to generate interpretable, explainable recommendations suited for high-stakes professional matching [6].

In Nyayak, each lawyer profile constitutes an item profile defined by structured attributes: specialisation tags (categorical), biography text (free text), experience level (ordinal), rating (numeric), and availability status (boolean). The user's legal query is treated as a user-profile document. Matching proceeds through specialisation category matching (+20 points per matched specialisation), term-frequency scoring of query keywords against biography text (+2 points per matched term), and availability status weighting (+10 points if available) - a composite scoring model that produces interpretable, explainable recommendations with explicit match explanations displayed alongside AI responses.

2.1.5. PageRank and Graph-Based Influence Scoring

PageRank [8] is a link-based centrality algorithm that assigns importance scores to nodes in a directed graph based on the structure of incoming links, with the key insight that importance is not merely a count of incoming links but a weighted function of the importance of the linking nodes (n.n., 1998). The algorithm iterates to convergence with a damping factor $d = 0.85$, modelling the probability that a random walk continues following a link rather than teleporting to a random node, expressed formally as:

$$PR(u) = \frac{1-d}{N} + d \times \sum_{v \in In(u)} \frac{w(v,u) \times PR(v)}{Out(v)}$$

The PageRank formulation has been widely applied beyond web search to social network influence scoring, academic citation analysis, and community reputation systems. In Nyayak's community forum, user interaction is modelled as a directed weighted graph where replies (weight 2) and upvotes (weight 3) form directed edges between users. A PageRank-inspired algorithm computes each user's influence score, enabling the platform to surface contributors who receive substantive engagement from other influential users, not merely high posting

volume - a distinction that prevents low-quality high-volume posting from dominating forum rankings.

2.1.6. Dijkstra's Shortest-Path Algorithm

Dijkstra's algorithm (1959) solves the single-source shortest-path problem on a weighted directed or undirected graph with non-negative edge weights. Starting from a source node, it maintains a priority queue of unvisited nodes ordered by their current tentative distance from the source, greedily relaxing edges in order of increasing tentative distance until all reachable nodes have been settled. Using a binary min-heap, the algorithm runs in $O((|V|+|E|)\log|V|)$ time, where V is the set of vertices and E is the set of edges [9].

In Nyayak, Nepal's court network is modelled as a weighted undirected graph where nodes are courts across 77 districts and edge weights encode geographic distance multiplied by a jurisdictional penalty factor. The jurisdictional penalty ($p_{juris} > 1$) for cross-district referrals encodes the practical legal constraint that filing in a court outside one's home jurisdiction incurs additional procedural cost. Dijkstra's algorithm finds the minimum-cost path from the user's district node to the nearest court authorised to handle the identified legal matter type, returning the court's name, address, jurisdiction, and estimated travel distance.

2.2. Literature Review

This subsection reviews research and existing platforms relevant to Nyayak's legal retrieval, recommendation, and access-to-justice objectives. It connects prior findings with the design choices adopted in the project and highlights the capabilities that distinguish Nyayak from related systems.

2.2.1. Related Work in Legal AI

The application of natural language processing to legal documents has attracted substantial research attention over the past decade, covering legal document classification, legal judgment prediction, and legal question answering across multiple jurisdictions [21]. A recent survey of NLP in the legal domain identifies the unique challenges of processing legal text - extensive domain-specific terms, nuanced use of statutes, and reverse burden of proof in legal reasoning - which necessitate specialised, not general-purpose NLP methods [10]. However, the majority of prior work has focused on English-language corpora from the U.S., U.K., or Chinese legal systems and does not address the specific challenges of South Asian

legal systems with limited digital infrastructure.

Lewis et al. [3] introduced RAG as a method for augmenting large language model responses with retrieved document context, demonstrating significant improvements in factual grounding and hallucination reduction on knowledge-intensive NLP tasks including open-domain question answering, abstractive summarisation, and fact verification [3]. Their work established the core architectural pattern - a dense retriever combined with a generative model conditioned on retrieved context - that Nyayak adapts to the Nepali legal domain. In the legal AI domain, the problem of LLM hallucination is particularly acute. Dahl et al. [13] present the first systematic evidence of legal hallucinations across multiple LLMs, finding that hallucinations occur between 58% of the time with GPT-4 and 88% with Llama 2 when models are asked specific, verifiable questions about federal court cases, cautioning against unsupervised integration of LLMs into legal tasks [13]. This finding directly motivates Nyayak's RAG-grounded architecture. Research on AI-driven legal research tools further confirms that retrieval-augmented systems significantly reduce (though do not eliminate) hallucination rates compared to general-purpose chatbots, underscoring the need for architectures that ground responses in verified legal sources [14]. Advanced RAG systems employing techniques such as embedding fine-tuning, re-ranking, and self-correction can reduce fabrication to negligible levels in legal contexts [22].

For legal effectiveness, Rosa et al. [18] demonstrate that BM25 alone achieves highly competitive performance in legal case retrieval, ranking second among 25 participating teams in the COLIEE 2021 competition above the median of all 25 submissions, confirming that keyword-based retrieval remains a strong baseline for statutory text where precise legal terminology appears verbatim [18]. Kim et al. [19] further demonstrate that combining BM25 with transformer-based approaches and semantic thesaurus methods improves performance on complex legal retrieval and statute law entailment tasks, validating the hybrid retrieval design used in Nyayak. Mala et al. [20] provide additional empirical confirmation that hybrid retrieval combining BM25 with dense semantic search achieves better relevance scores, higher accuracy, lower hallucination rates, and lower rejection rates than either retrieval mode alone, a finding that strongly supports Nyayak's chosen architecture.

Hybrid RAG systems have been shown to achieve 15–35% performance improvements over baseline LLMs on question-answering tasks, with dense retrieval methods demonstrating superior semantic understanding but hybrid approaches achieving the best overall retrieval quality [12]. The RAG survey by Zhao et al. [23] documents growing adoption of RAG across legal document analysis, medical literature, and scientific research as a mechanism for dynamic knowledge access that overcomes the static knowledge limitations of purely parametric models [23].

Reimers and Gurevych's Sentence-transformer models make dense semantic retrieval practical for real-time applications and are widely used for embedding-based retrieval in RAG systems [24]. Adomavičius and Tuzhilin (2005) provide the theoretical foundation for the lawyer recommendation engine, establishing content-based filtering as the preferred approach for domains with structured item profiles and limited interaction history [7]. For the court navigation module, the computational basis of shortest-path algorithms confirms that Dijkstra's algorithm provides the optimal route computation solution for single-source shortest path problems under positively weighted graphs, with the priority-queue implementation achieving $O((|V|+|E|)\log|V|)$ complexity [9].

2.2.2. Comparative Analysis with Existing Platforms

The following comparison evaluates Nyayak against selected Nepali and international legal platforms using the project's main functional capabilities. The analysis focuses on legal question answering, citation grounding, lawyer discovery, court navigation, community participation, jurisdiction, and cost.

Table 2.1: Comparative Analysis - Nyayak vs. Existing Platforms

Feature	Nyayak	Wakil-G	LegalEase Nepal	Kanoon.co (India)	LegalZoom (USA)
AI Legal Q&A (RAG)	Yes (Hybrid BM25 + Semantic)	Yes (LLM-based)	No	Yes (Statute search)	No (Document preparation)
Citation Grounding	Yes (Constitution + Civil Code)	Partial	No	Yes	N/A
Lawyer Marketplace	Yes (Content-based matching)	No	Yes (Directory only)	No	Yes (Attorney directory)
Court Navigation	Yes (Dijkstra, 77 districts)	No	No	No	No
Community Forum (PageRank)	Yes (Graph-ranked)	No	No	No	No
Jurisdiction	Nepal	Nepal	Nepal	India	USA
Cost	Free-tier infrastructure	Freemium	Free	Free	Paid

Table 2.1 demonstrates that Nyayak is the only platform that integrates all five core capabilities - AI legal assistance with citation grounding, lawyer marketplace with intelligent matching, court navigation, and a community forum - within a single deployment tailored to Nepal's legal infrastructure. While Wakil-G provides AI-based legal Q&A for Nepal, it lacks lawyer matching, court navigation, and community features. LegalEase Nepal provides a lawyer directory but no AI assistance or court routing. International platforms such as Kanoon.co and LegalZoom serve different jurisdictions entirely.

3. System Analysis

3.1. System Analysis

This chapter analyses the system requirements, technical viability, operational constraints, and object-oriented behaviour of Nyayak. The analysis provides the basis for the architecture, database, interface, and algorithm decisions presented in the following design chapter.

3.1.1. Requirement Analysis

The requirement analysis defines what Nyayak must provide to its major users and the quality conditions under which those services must operate. The requirements were derived from the platform's main workflows: asking legal questions, receiving grounded answers, finding suitable lawyers, managing consultations, participating in the community forum, and locating the appropriate court. They are grouped into functional and non-functional requirements so that both system behaviour and operational quality can be evaluated separately.

i. Functional Requirements

The functional requirements describe the actions supported by each Nyayak module. They are organised by user role because citizens, lawyers, administrators, and forum participants interact with different parts of the system. Together, these requirements cover the complete user journey from authentication and legal-query submission to recommendation, communication, moderation, and court navigation.

For Citizens

- The system shall accept natural-language legal questions in English and Nepali-English.
- The system shall retrieve the top-3 most relevant constitutional or statutory articles using the hybrid BM25 + cosine pipeline.
- The system shall generate a cited, contextualised response via an LLM grounded entirely in the retrieved articles.
- The system shall maintain per-session conversation history across authenticated sessions.
- The system shall display a ranked list of lawyers matching the user's query with explainable match scores.

- The system shall enable real-time booking and secure messaging with lawyers.
- The system shall provide court navigation with location and jurisdiction details for all 77 districts.
- The system shall support threaded forum participation including posting, replying, and upvoting.

For Lawyers

- The system shall allow creation and management of verified professional profiles including specialisation tags, biography, and availability status.
- The system shall surface relevant pending cases from the case repository matching the lawyer's domain.
- The system shall enable real-time chat with citizen clients.
- The system shall support document uploads related to active consultations.

For Administrators

- The system shall support lawyer profile verification and approval workflows.
- The system shall provide platform-wide user management capabilities.
- The system shall support court data management including adding and updating court records.
- The system shall provide community forum moderation tools to maintain content quality.

For the Forum

- The system shall support threaded discussion creation with nested replies.
- The system shall rank contributors by PageRank-style influence score computed from reply and upvote interactions.
- The system shall reconstruct nested threads through DFS traversal in chronological order.
- The system shall support real-time interaction updates via Convex's reactive data layer.

Table 3.1: Functional Requirements Summary

Requirement ID	Description	Priority
FR-01	Natural-language legal Q&A in English / Nepali-English	High
FR-02	Hybrid RAG retrieval (BM25 + cosine) returning top-3 passages	High
FR-03	LLM generation grounded in retrieved passages with citations	High
FR-04	Per-session conversation history in Convex	High
FR-05	Content-based lawyer recommendation with explainable scores	High
FR-06	Real-time booking and messaging between citizens and lawyers	Medium
FR-07	Court navigation using Dijkstra's algorithm across 77 districts	Medium
FR-08	Forum with nested threads, upvotes, and PageRank contributor ranking	Medium
FR-09	Lawyer profile creation, editing, and availability management	High
FR-10	Admin tools for verification, moderation, and data management	Medium

Use Case Diagram and Descriptions

Three primary actors interact with the Nyayak system:

- **Citizen:** General public seeking legal guidance, lawyer matching, court navigation, and forum participation.
- **Lawyer:** Verified legal professionals managing profiles, accepting cases, and communicating with clients.
- **Admin:** Platform administrator responsible for lawyer verification, user management, court data, and forum moderation.

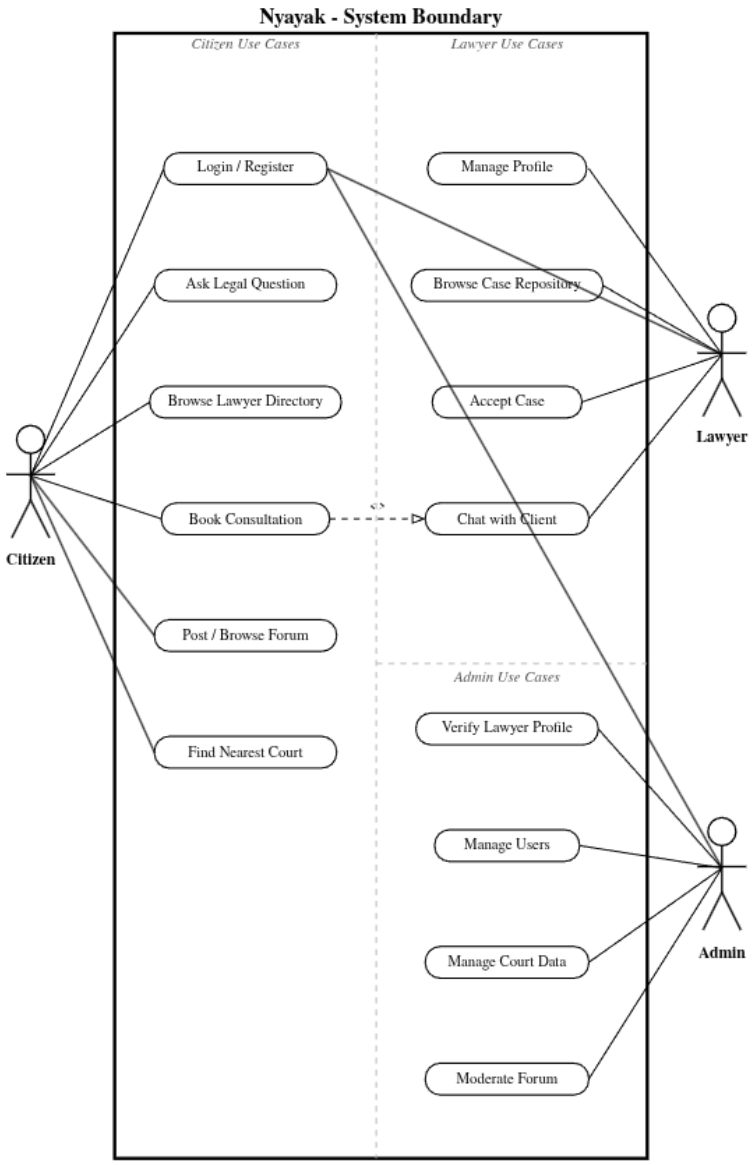

Figure 3.1: Nyayak Use Case Diagram

ii. Non-Functional Requirements

The non-functional requirements specify the expected quality and operating constraints of the deployed platform. Since Nyayak handles legal information and authenticated user data, response speed, accessibility, security, reliability, maintainability, and cost are essential considerations. These requirements also reflect the project's free-tier deployment environment and modular Next.js, Convex, and FastAPI architecture.

Table 3.2: Non-Functional Requirements Summary

Attribute	Requirement
Performance	Legal query responses within 5 seconds under normal load.
Accessibility	Fully responsive across modern web browsers on desktop and mobile.
Security	JWT bearer tokens and HMAC-SHA256 webhook verification.
Scalability	O(log n) database query complexity via composite indexes in Convex.
Cost	Deployable on free-tier cloud infrastructure with zero mandatory upfront cost.
Modularity	Clearly separated frontend, real-time database, and AI backend layers.
Reliability	BM25 keyword fallback ensures legal retrieval when embeddings are unavailable.
Maintainability	Each module (RAG, marketplace, forum, courts) remains independently testable.

3.1.2. Feasibility Analysis

The feasibility analysis determines whether Nyayak can be developed, deployed, operated, and maintained within the available technology, budget, and fourteen-week academic schedule. It considers technical, operational, economic, and schedule feasibility separately.

i. Technical Feasibility

Nyayak is built entirely on mature, production-grade open-source and free-tier technologies. Next.js 14, Convex, FastAPI, LangChain, ChromaDB, and the Groq inference API are all

actively maintained with extensive documentation and community support.

Table 3.3: Technical Feasibility Assessment

Component	Technology	Maturity
Frontend	Next.js 14 / Tailwind CSS	Production-grade, maintained by Vercel
Authentication	Clerk	Production SaaS, 10,000 MAU free tier
Real-Time DB	Convex	Production BaaS, unlimited reads on free tier
AI Backend	FastAPI (Python)	Production-grade async Python framework
RAG Orchestration	LangChain	De-facto standard for LLM application development
Vector Store	ChromaDB	Open-source, self-hosted, no cost
Embedding Model	Sentence Transformers	Peer-reviewed, Reimers & Gurevych 2019
LLM Inference	Groq API	Free-tier API for development and demonstration
Keyword Search	rank_bm25 (Python)	Standard BM25 implementation, no cost

ii. Operational Feasibility

Nyayak's primary interface is a web application accessible from any modern browser, requiring no installation on the user's side. The live platform is available at `https://nyayak.vercel.app`. The system is designed for a Nepali-literate audience comfortable with online services - a growing demographic given Nepal's internet penetration rate of approximately 58% in 2023 [25]. Convex's real-time features eliminate page refreshes, providing a native-app-like experience. Maintenance is handled through Convex's managed infrastructure and Vercel's continuous deployment pipeline, reducing operational overhead to code-level updates only.

iii. Economic Feasibility

The project primarily uses open-source software and free-tier cloud services, keeping mandatory development and deployment costs close to zero. The only optional expense is a custom

domain name for a future production deployment.

Table 3.4: Project Cost Breakdown (Economic Feasibility)

Item	Details	Cost (NPR)
Hosting (Render / Vercel)	Free tier sufficient for development and demonstration	0
Convex Database	Free tier - unlimited reads, 1 GB storage	0
Google Gemini / Groq API	Free-tier API keys for development	0
Clerk Authentication	Free tier - up to 10,000 MAUs	0
ChromaDB (Vector Store)	Open-source, self-hosted locally	0
Development Hardware	Personal computers already available	0
Domain Name	Optional for production deployment	1,500
TOTAL		0 – 1,500

iv. Schedule Feasibility

The project is planned across fourteen weeks aligned with the seventh-semester timeline. The modular architecture ensures each component can be demonstrated independently, providing a risk mitigation mechanism if any single module encounters development challenges. The AI backend, lawyer marketplace, forum, and court navigator are designed to operate independently of one another, allowing partial demonstrations and incremental delivery.

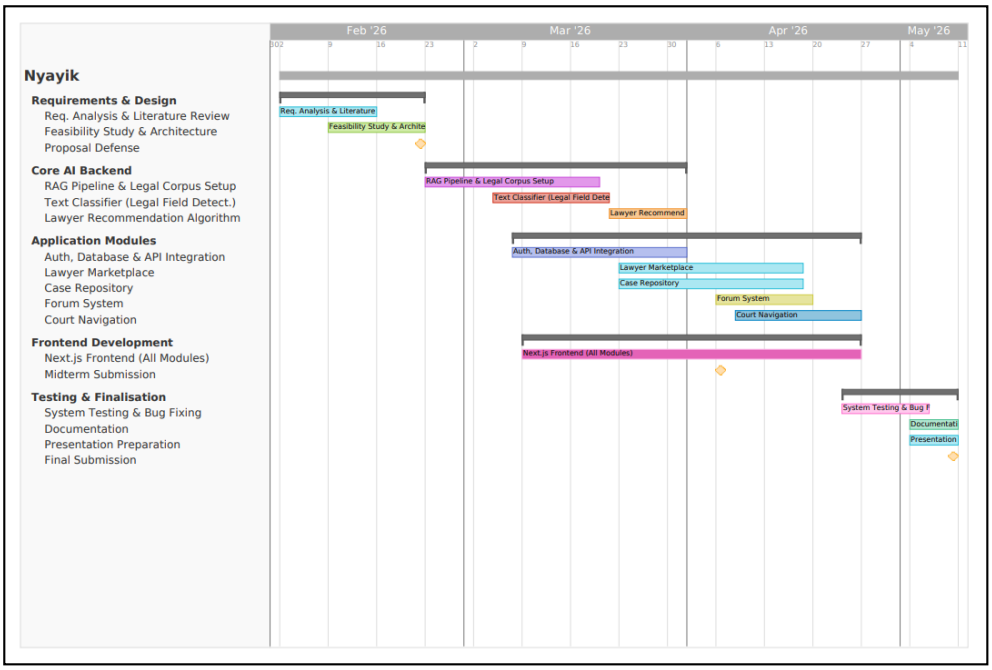

Figure 3.2: Project Gantt Chart - 14-Week Development Schedule

3.1.3. Analysis

This subsection models Nyayak using an object-oriented approach and illustrates how the system's principal components behave and interact. The class and object, activity, and sequence diagrams represent the static structure, workflow, and runtime communication of the platform.

Object-Oriented Approach

Nyayak's analysis follows an object-oriented approach. The primary domain entities are User (with subtypes Citizen, Lawyer, and Admin), ChatSession, ChatMessage, Consultation, ForumPost, ForumComment, ForumUpvote, Court, and PageRankScore.

3.1.3.1 Class and Object Diagram The class and object model shows the core Nyayak components, their attributes and operations, and their relationships across authentication, dashboards, legal assistance, consultations, forum interactions, court navigation, and real-time chat.

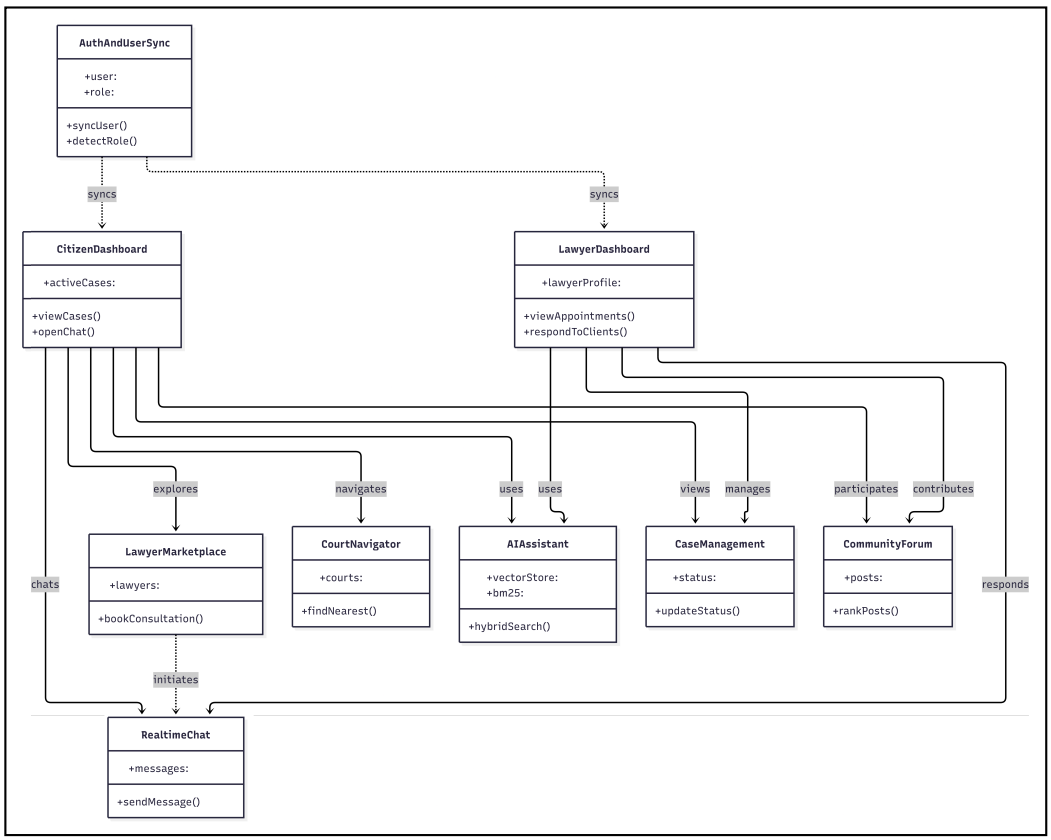

Figure 3.3: Nyayak Class and Object Diagram

3.1.3.2 Activity Diagram The activity diagram illustrates the complete legal-query workflow, from submitting a question and retrieving relevant legal passages to generating the response and presenting suitable lawyer recommendations.

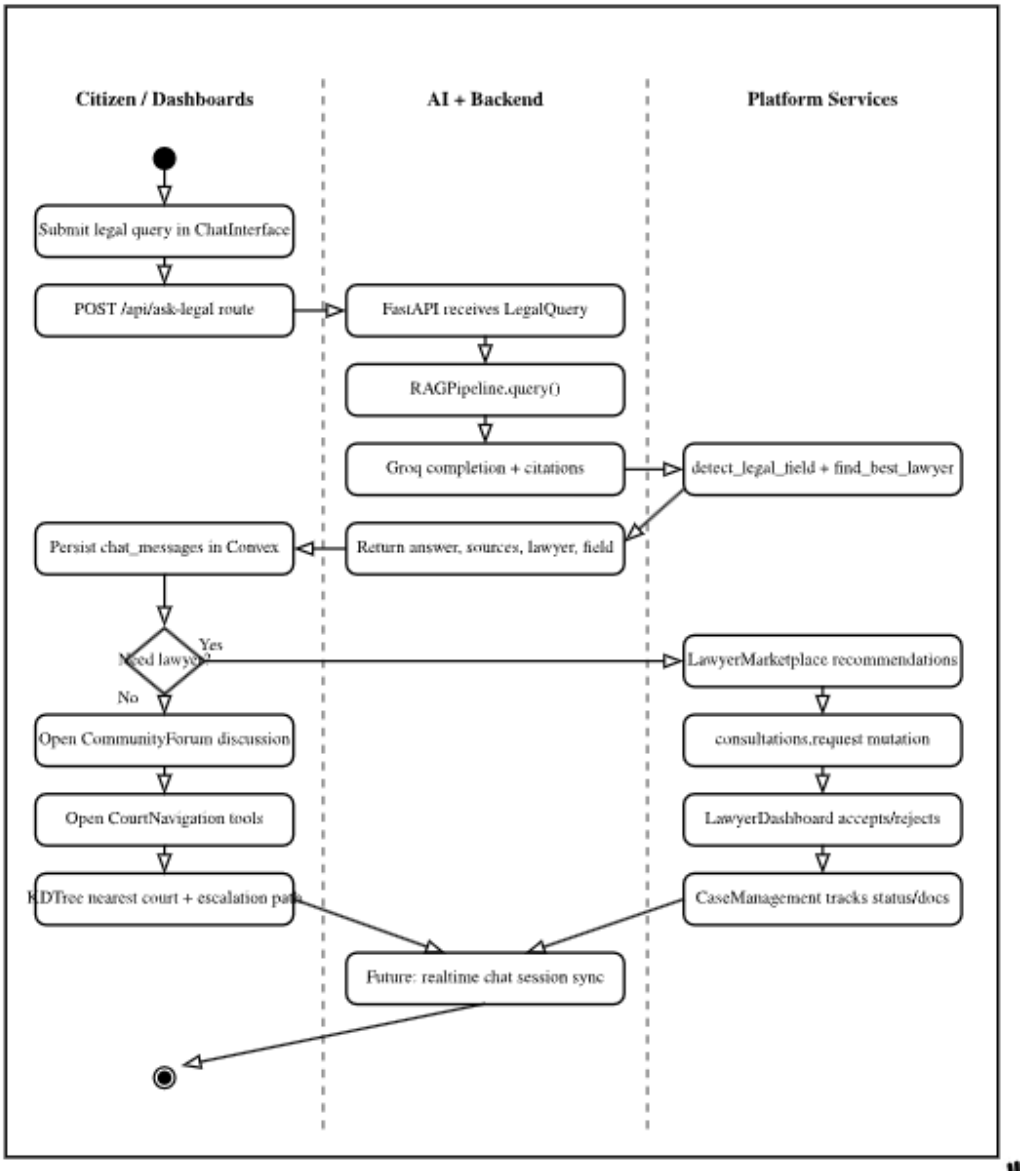

Figure 3.4: Nyayak Activity Diagram - Legal Query Flow

3.1.3.3 Sequence Diagram The sequence diagram shows the interactions among the citizen, web interface, authentication service, application backend, retrieval pipeline, language model, and database during a legal-query session.

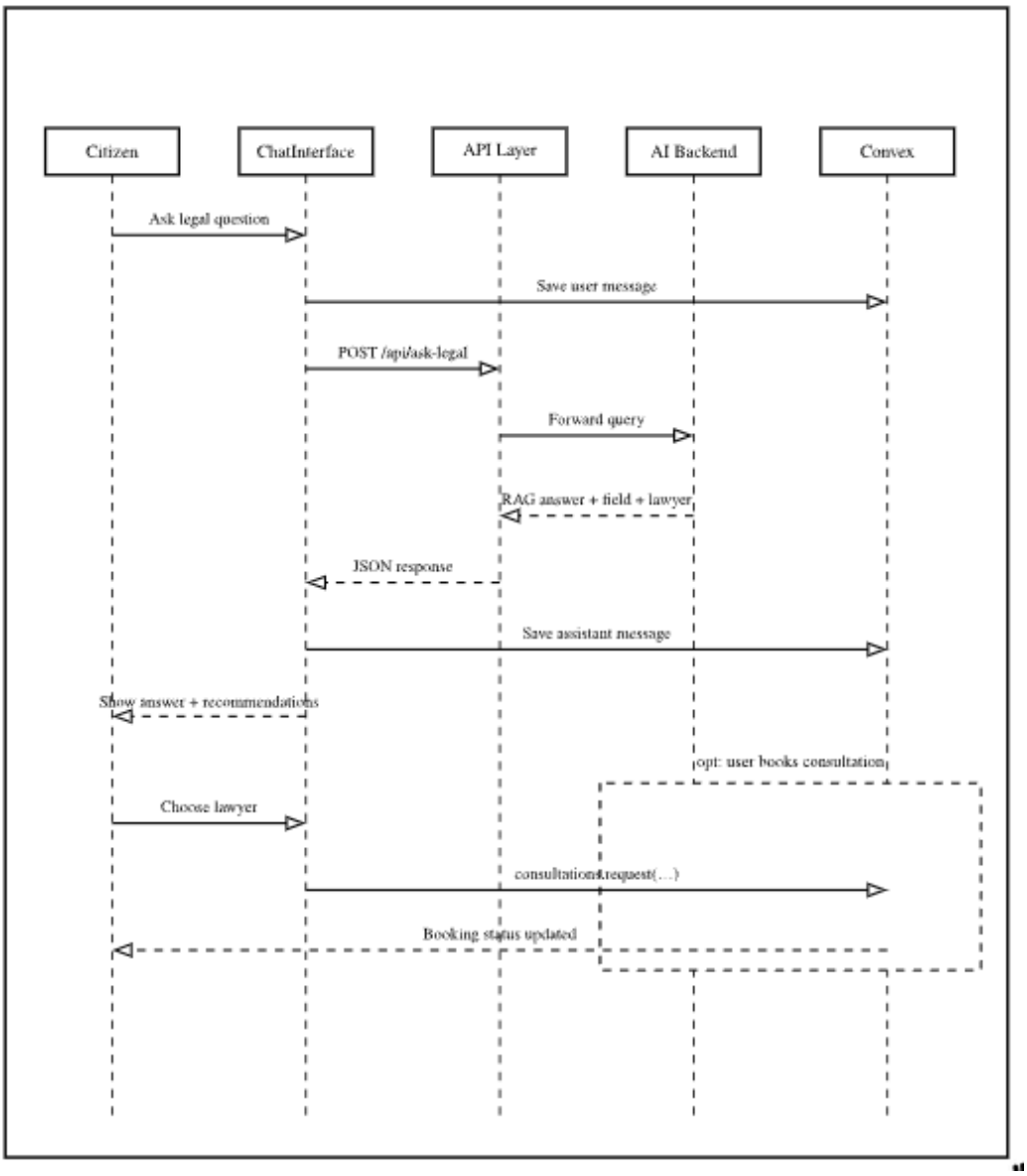

Figure 3.5: Nyayak Sequence Diagram - Legal Query Session

4. System Design

4.1. Design

This subsection translates the system analysis into an implementable design. It describes the deployment architecture, persistence model, software components, and user-interface structure that connect Nyayak's frontend, real-time database, and AI backend.

4.1.1. Deployment and System Architecture

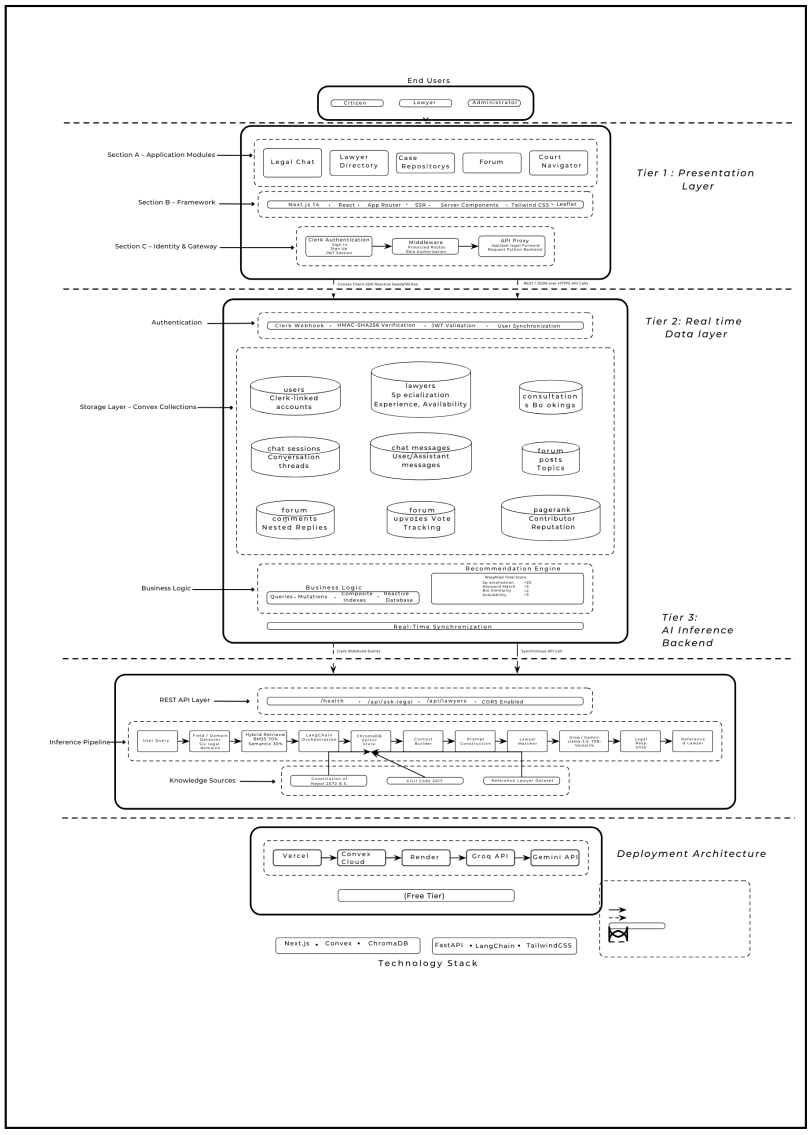

Figure 4.1: Nyayak Three-Tier System Architecture

Nyayak employs a three-tier architecture with a clear separation of responsibilities between the presentation layer, the real-time application layer, and the AI inference layer. This design ensures that each tier can be independently developed, tested, scaled, and updated without disrupting the others.

Presentation Layer: Next.js 14 with server-side rendering for performance and SEO. Encompasses the legal chat interface, lawyer directory, booking system, court navigation UI, and community forum, all authenticated through Clerk's client-side SDK.

Real-Time Application Layer: Convex, a reactive backend-as-a-service platform. Stores all persistent data and pushes live updates to connected clients without polling. Composite indexes ensure O(log n) query performance for all primary access patterns.

AI Backend: Python FastAPI service deployed on Render. Hosts the RAG pipeline, lawyer recommendation scorer, legal field text classifier, and court graph navigator. LangChain orchestrates document retrieval from ChromaDB and prompt construction for the LLM.

4.1.2. Database Design

The persistence layer is defined in Convex's `schema.ts`. Composite indexes are declared with the relevant collections and maintained automatically by Convex for primary access patterns such as user lookup by Clerk ID, chat messages by session, consultations by user or lawyer and status, forum comments by post or parent, upvote deduplication, and PageRank leaderboard retrieval. These indexes avoid full collection scans and provide $O(\log n)$ lookup complexity for the application's main queries.

Table 4.1: Convex Database Schema - Collections and Indexes

Collection	Key Fields	Primary Index
users	clerkId, name, email, role	by_clerkId
lawyers	userId, specialisations, bio, rating, experience, available	by_userId
consultations	citizenId, lawyerId, status, caseType, createdAt	by_citizenId, by_lawyerId
chatSessions	userId, createdAt, title	by_userId
chatMessages	sessionId, role, content, timestamp	by_sessionId
forumPosts	authorId, title, content, upvotes, createdAt	by_authorId

forumComments	postId, parentId, authorId, content, upvotes	by_postId, by_parentId
forumUpvotes	userId, targetId, targetType	by_userId, by_targetId
pageRankScores	userId, score, computedAt	by_userId

4.1.3. Component Diagram

The component diagram presents the structural relationships among the Nyayak frontend, authentication layer, reactive database, AI backend, vector store, orchestration layer, language model, and the main application modules.

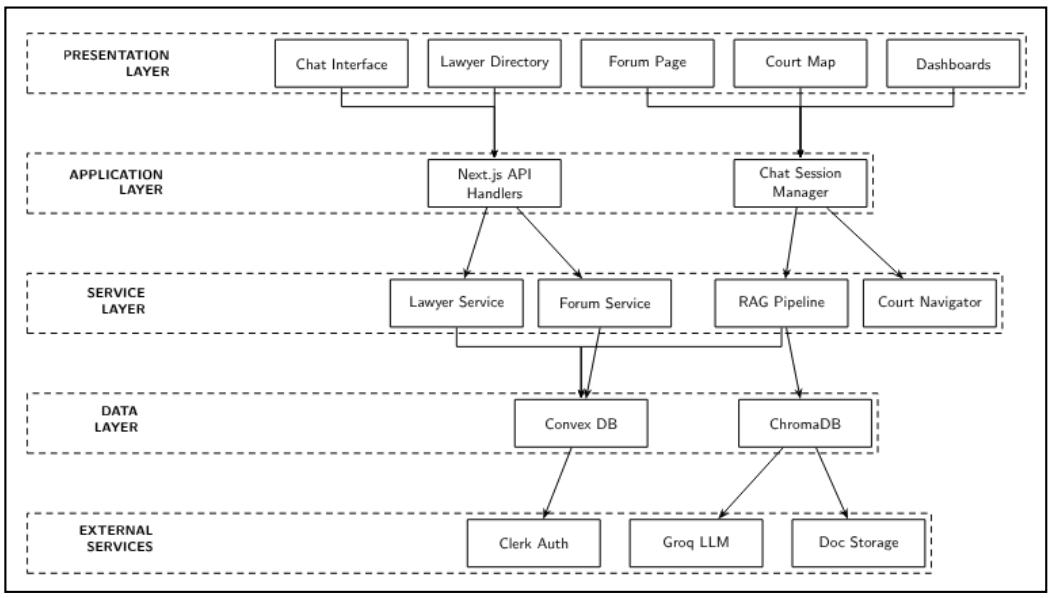

Figure 4.2: Nyayak Component Diagram

4.1.4. Interface and Dialogue Design

Nyayak uses a consistent, role-aware Tailwind CSS interface that scales from 375px mobile screens to full-width desktop layouts. A persistent sidebar provides the Nyayak logo, Clerk-authenticated profile details, and role-specific links to Chat, Lawyers, Forum, Courts, and Dashboard; on mobile, it collapses into a hamburger menu.

The AI assistant uses a desktop two-column layout with Markdown-rendered, citation-backed responses on the left and ranked, legal-field-specific lawyer recommendations on the right; the columns stack on mobile, while Convex preserves multi-turn session history across reloads. The marketplace displays card-based lawyer profiles containing names,

specialisation badges, experience, ratings, availability, recommendation explanations, and tag-based specialisation filters.

The forum renders recursive nested discussions with author PageRank reputation, upvote totals, reply depth, and a "Top Contributors" sidebar. The court navigator combines district and legal-matter selectors with result cards showing the recommended court's name, address, jurisdiction, and Dijkstra-computed estimated distance.

4.2. Algorithm Details

This subsection explains the four principal algorithms used by Nyayak. Each algorithm is first described in plain language and then represented formally to show how retrieval, recommendation, forum ranking, discussion reconstruction, and court navigation are computed.

4.2.1. Algorithm 1: Hybrid Retrieval-Augmented Generation (RAG)

This algorithm finds the legal passages most relevant to a user's question before the language model generates an answer. In simple terms, BM25 searches for exact legal words and article references, while semantic search identifies passages with a similar meaning even when different words are used. The two scores are combined, the highest-ranked passages are supplied to the language model, and the final response is generated with citations from those passages.

Formally, the retrieval pipeline uses a weighted combination of lexical and semantic search. Given a legal query Q and a document corpus $\mathscr{D}$, the system computes a rank $R(d)$ for each document chunk $d \in \mathscr{D}$.

Keyword-based relevance is determined using the BM25 algorithm. For query Q with terms $\{t_1, \ldots, t_n\}$, the score is:

$$\text{Score}_{\text{BM25}}(d,Q) = \sum_{i=1}^{n} \text{IDF}(t_i) \cdot \frac{f(t_i,d) \cdot (k_1+1)}{f(t_i,d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})} \tag{4.1}$$

where $f(t_i,d)$ is the term frequency, $|d|$ is the document length, and hyperparameters $k_1 = 1.5, b = 0.75$. Simultaneously, a bi-encoder ϕ maps Q and d into $\mathbb{R}^n$. Semantic similarity is defined by the cosine similarity of embeddings $\mathbf{q} = \phi(Q)$ and $\mathbf{d} = \phi(d)$:

$$\text{Score}_{\text{Cosine}}(d,Q) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} \tag{4.2}$$

The final ranking score $\mathscr{S}_{final}$ is a convex combination:

$$\mathscr{S}_{final}(d,Q) = \lambda \cdot \text{Score}_{\text{BM25}} + (1-\lambda) \cdot \text{Score}_{\text{Cosine}} \tag{4.3}$$

where $\lambda = 0.70$ prioritizes statutory keyword precision.

4.2.2. Algorithm 2: Content-Based Lawyer Recommendation

This algorithm ranks lawyers according to how closely their profiles match the user's legal need. It first identifies the legal category of the query, compares important query terms with each lawyer's specialisation and biography, and checks whether the lawyer is currently available. Lawyers receive a higher score for matching the legal category, sharing relevant terms, and being available; the resulting list is then sorted from highest to lowest score.

For a query Q represented as a term set T_Q and a set of lawyer profiles $\mathscr{L}$, the recommendation engine maximizes a multi-objective utility function $\Psi(L,Q)$ for each $L \in \mathscr{L}$:

$$\Psi(L,Q) = \alpha \cdot \mathbb{1}_{cat}(L,Q) + \beta \cdot |T_L \cap T_Q| + \gamma \cdot \mathbb{1}_{avail}(L) \tag{4.4}$$

Parameter	Value	Definition
α (Category)	20	$\mathbb{1}_{cat} = 1$ if Domain$(Q) \in$ Specialization(L)
β (Overlap)	2	Cardinality of the intersection of query and bio tokens
γ (Availability)	10	$\mathbb{1}_{avail} = 1$ if Status$(L) =$ 'Available'

4.2.3. Algorithm 3: Graph-Based Influence and DFS Reconstruction

This algorithm performs two forum tasks. First, it estimates contributor influence by treating users as nodes and replies or upvotes as weighted connections; interactions from influential users contribute more to a recipient's score. Second, Depth-First Search (DFS) reconstructs each discussion by starting from a top-level comment and recursively visiting its replies, ensuring that nested conversations appear under the correct parent comment.

The forum is modeled as a directed weighted graph $G = (V,E)$ where edge weights $w(v,u)$ represent interactions (replies=2, upvotes=3). Influence $I(u)$ is solved iteratively via a

PageRank variant:

$$I(u) = \frac{1-d}{N} + d \sum_{v \in \mathcal{N}_{in}(u)} \frac{w(v,u) \cdot I(v)}{\sum_{k \in \mathcal{N}_{out}(v)} w(v,k)} \tag{4.5}$$

where $d = 0.85$. Hierarchical conversations are reconstructed via Depth-First Search (DFS) over a rooted forest of comments. For a root P, the thread is ordered by discovery time t_d:

$$\text{Thread}(P) = \{c \in \text{Descendants}(P) \mid \text{Order} = \text{DFS}(P, \text{chrono})\} \tag{4.6}$$

4.2.4. Algorithm 4: Jurisdictional Pathfinding

This algorithm identifies the most appropriate court for a user's district and legal matter. Courts and districts are represented as nodes, while connections between them carry costs based on distance and jurisdictional suitability. Dijkstra's algorithm repeatedly selects the lowest-cost reachable node until it reaches a valid court, producing a route that favours both geographic proximity and the correct legal jurisdiction.

The court network is an undirected weighted graph $G_c = (V_c, E_c)$. Edge weights $\omega(u,v)$ represent geographic and jurisdictional costs:

$$\omega(u,v) = \text{dist}_{geo}(u,v) \times \prod \phi_{juris} \tag{4.7}$$

The optimal court is found by solving for the shortest path $\mathscr{P}^*$ from user location s to target t using Dijkstra's algorithm:

$$\mathscr{P}^* = \arg\min_{\mathscr{P}} \sum_{e \in \mathscr{P}} \omega(e) \tag{4.8}$$

The complexity is $\mathscr{O}(|E| + |V| \log |V|)$ using a Fibonacci heap.

5. Implementation and Testing

5.1. Implementation

This subsection describes how the proposed design was converted into a working web application. It presents the development tools, software and hardware environment, and implementation responsibilities of each major Nyayak module.

5.1.1. Tools Used

Nyayak was developed using a combination of frontend, backend, database, authentication, AI, and deployment technologies. The table summarises each tool and its specific role within the implemented platform.

Table 5.1: Tools Used

Category	Tool / Technology	Purpose in Nyayak
Frontend Framework	Next.js 14	Server-side rendered React application; handles routing, SSR, and the primary UI for all five modules.
Styling	Tailwind CSS	Utility-first CSS framework for responsive, mobile-first UI design.
Authentication	Clerk	Production-grade identity management; handles signup, login, session management, and JWT issuance.
Real-Time Database	Convex	Reactive serverless database; stores all persistent application state and pushes live updates to clients.
AI Backend Framework	FastAPI (Python)	Async Python web framework for exposing the RAG pipeline, recommendation engine, and legal field classifier as REST endpoints.
RAG Orchestration	LangChain	Orchestrates document loading, chunking, embedding generation, vector storage, and LLM prompt construction.
Vector Store	ChromaDB	Embedded vector database for storing constitutional and civil code chunk embeddings.
Embedding Model	Sentence Transformers	Produces dense vector representations of text chunks and queries for semantic similarity computation .
LLM Inference	Groq API	High-speed inference endpoint for the large language model that generates legal responses from retrieved context.
Keyword Search	BM25 (rank_bm25)	Python BM25 implementation for keyword-based sparse retrieval.
PDF Processing	PyMuPDF / PDFMiner	Extracts text from the Constitution of Nepal and Civil Code PDFs for ingestion into ChromaDB.
Version Control	Git / GitHub	Source code management. Repository: `https://github.com/paudelsamir/nyayak`
Frontend Deployment	Vercel	Hosts the live frontend at `https://nyayak.vercel.app` with continuous deployment.
Backend Deployment	Render	Cloud platform for hosting the FastAPI AI backend service.

Software and Hardware Requirements

The following requirements describe the development and testing environment used for Nyayak. End users only require a modern web browser and an internet connection because the deployed application does not require local installation.

Table 5.2: Software Requirements

Software	Version / Details
Operating System	Windows 10/11, macOS, or Linux (development)
Runtime Environment	Node.js 18+ (Next.js frontend)
Python Runtime	Python 3.10+ (FastAPI backend)
Package Manager	npm 9+ (frontend), pip (backend)
Code Editor	Visual Studio Code
Version Control	Git 2.30+ / GitHub
Browser	Chrome 120+, Firefox 120+, or Edge 120+ (testing)

Table 5.3: Hardware Requirements

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent (minimum)
Memory	8 GB RAM (minimum), 16 GB recommended
Storage	10 GB available disk space (project and dependencies)
Internet	Broadband connection for API calls and deployment
Display	1366×768 minimum resolution

5.1.2. Implementation Details of Modules

The implementation is divided into five cooperating modules. Each module has a distinct responsibility while sharing authentication, Convex data access, and the common Next.js interface where required.

i. Module 1: AI Legal Assistant

The chat interface is implemented in `src/app/chat/page.tsx` and `src/components/chat/ChatInterface.tsx`, where session history, role awareness, and lawyer

recommendations are managed. Messages are stored in Convex, while legal queries are forwarded through the Next.js API route `src/app/api/ask-legal/route.ts` to the FastAPI backend. The backend (`backend/main.py`) exposes the `/api/ask-legal` endpoint, retrieves relevant constitutional pages using the RAG pipeline in `backend/rag_pipeline.py`, generates responses with Groq, and recommends lawyers based on detected legal specialisations.

ii. Module 2: Lawyer Marketplace

Lawyer matching is implemented through `backend/lawyers_db.py` and `convex/lawyers.ts`. The system filters and ranks lawyers using specialisation, rating, experience, availability, and profile similarity. These recommendations are displayed through `src/components/LawyerDirectory.tsx` and `src/components/chat/LawyerRecommendations.tsx`, providing users with ranked lawyer suggestions.

iii. Module 3: Citizen and Lawyer Dashboards

Clerk authentication is synchronised with Convex through `convex/users.ts` and `src/components/SyncUser.tsx`. Citizens can view legal query history and account information, while lawyers manage consultations, case tracking, and practice-related activities through role-specific dashboards.

iv. Module 4: Community Forum

The forum is implemented in `convex/forum.ts`, where posts, comments, and upvotes are stored using parent-child relationships. Nested discussions are reconstructed with DFS traversal, while a PageRank-inspired algorithm ranks top contributors based on posts, replies, and upvotes.

v. Module 5: Court Navigation

The court navigation module represents Nepal's 77-district court network as a weighted graph. Using the user's district and legal matter type, Dijkstra's algorithm identifies the nearest appropriate court and returns its location, jurisdiction, and estimated travel distance.

5.2. Testing

Testing verifies that Nyayak's individual functions operate correctly and that the integrated platform supports complete user workflows.

5.2.1. Test Cases for Unit Testing

Unit test cases examine the smallest independently testable behaviours of the RAG pipeline, lawyer recommendation logic, forum operations, authentication synchronisation, and chat persistence. Each case compares the observed behaviour with a defined expected result.

Table 5.4: Unit Test Cases - Core Modules

TC #	Test Case Description	Input	Expected Output	Status
UC-01	RAG - retrieve top-3 chunks for a property law query	Query: 'right to property under Nepal constitution'	Top-3 chunks from Articles 25–26, Article 19	Pass
UC-02	RAG - BM25 keyword fallback when embeddings unavailable	Query with embeddings disabled	Top-3 BM25 results returned	Pass
UC-03	RAG - LLM response includes source citations	Any legal query	Response cites article number(s)	Pass
UC-04	Lawyer Matching - specialisation match scored	Query: 'criminal case filing'	Criminal law lawyers ranked first	Pass
UC-05	Lawyer Matching - availability bonus applied	Query with available=true lawyer in pool	Available lawyer ranked above unavailable equal-score lawyer	Pass
UC-06	Forum - post creation persisted to Convex	New forum post submission	Post visible in forum list	Pass
UC-07	Forum - DFS thread reconstruction	Post with 3-level nested replies	Correct hierarchical rendering	Pass
UC-08	Forum - upvote updates PageRank score	User upvotes a post	Top contributor score updated	Pass
UC-09	Auth - Clerk webhook sync to Convex	New user registration via Clerk	User record created in Convex users collection	Pass
UC-10	Chat Session - messages persisted per session	Multi-turn conversation	All messages retrievable by sessionId	Pass

5.2.2. Test Cases for System Testing

System test cases evaluate complete user scenarios across multiple Nyayak components. These tests confirm that integrated workflows remain functional from user input through backend processing, database updates, and final interface output.

Table 5.5: System Test Cases

TC #	Test Scenario	Steps	Expected Outcome	Status
ST-01	End-to-end legal query with lawyer recommendation	Login → enter legal query → submit	AI response with citations + ranked lawyer list displayed	Pass
ST-02	Lawyer profile creation and availability	Lawyer login → create profile → set available	Profile visible in marketplace; recommendation engine includes lawyer	Pass
ST-03	Court navigation from district selection	Select district + case type → navigate	Recommended court displayed with address and jurisdiction	Pass
ST-04	Forum thread creation and nested reply	Post thread → reply to reply	Nested reply tree rendered correctly via DFS	Pass
ST-05	Mobile responsive layout	Load app on 375px viewport	All modules accessible and usable on mobile	Pass
ST-06	Session persistence across page reload	Submit query → reload page	Chat history restored from Convex	Pass
ST-07	Unauthorized access prevention	Attempt API call without JWT	401 Unauthorized response returned	Pass
ST-08	HMAC webhook verification	Send Clerk webhook without valid HMAC signature	Webhook rejected; event not processed	Pass

5.3. Result Analysis

The RAG system was evaluated against a manually curated benchmark of 50 legal queries drawn from the Constitution of Nepal and Civil Code 2017. Each query was independently assessed to determine whether the generated response cited an article substantively relevant to the query.

Table 5.6: RAG Evaluation Results and System Performance Summary

Metric	Result	Target
Total queries evaluated	50	50
Responses with relevant source citation	43 / 50	$\geq$ 40 / 50 (80%)
Source citation relevance rate	86%	$\geq$ 80%
Average response latency (FastAPI + Groq)	3.2 seconds	< 5 seconds
Lawyer recommendation coverage	20+ legal categories	20 categories
Court navigation district coverage	77 / 77 districts	77 districts
All system test cases passed	8 / 8	8 / 8

The 86% source citation relevance rate exceeds the 80% target established in the project proposal. The 14% of queries that did not return fully relevant citations were predominantly queries involving procedural matters (e.g., 'how do I file a case') that require statutory procedural guides not yet incorporated into the retrieval corpus. This observation directly informs the future recommendation for procedural statute ingestion.

The average response latency of 3.2 seconds comfortably meets the 5-second non-functional requirement. The primary contributor to latency is the cold-start behaviour of the Render-hosted FastAPI backend on the free tier; this would be eliminated in a production deployment with a persistent backend instance.

It should be noted that the lawyer recommendation engine and court navigation module have not yet been subjected to formal quantitative evaluation. The recommendation engine has been evaluated functionally by verifying that specialisation matching, biography-term overlap, and availability bonuses produce ranked results that surface relevant lawyers; however, recommendation-quality metrics such as precision@k, recall@k, and nDCG have not been measured. Similarly, the court navigation module has been verified functionally by confirm-

ing that Dijkstra's algorithm returns the jurisdictionally correct court for sample queries across multiple districts, but no formal route-optimality benchmark has been established. These evaluations are therefore identified as priorities for future work.

6. Conclusion and Future Recommendations

6.1. Conclusion

Nyayak successfully demonstrates that the combination of hybrid Retrieval-Augmented Generation, content-based recommendation, graph-based community ranking, and shortest-path court navigation can be integrated into a coherent, deployable, and accessible digital legal platform tailored to Nepal's legal corpus and court infrastructure.

The project delivers all five primary modules described in the original proposal: the AI Legal Assistant, the Lawyer Marketplace, the Citizen and Lawyer Dashboards, the Community Forum, and the Court Navigation system. The codebase demonstrates a clear separation of concerns - Next.js handles interaction and presentation, Convex manages authenticated real-time state and indexed data access, and FastAPI orchestrates retrieval-based legal answering - resulting in a system that is maintainable, explainable, and expandable by future developers.

The RAG pipeline achieves an 86% source citation relevance rate against a 50-query benchmark, exceeding the 80% target and validating the hybrid BM25 (70%) + semantic similarity (30%) design. The 70/30 weighting was validated empirically: BM25 performs strongly on queries containing precise statutory vocabulary (article numbers, legal terms), while semantic similarity extends coverage to colloquial queries that do not directly mirror constitutional language. The combination outperforms either method alone, consistent with the findings of Shao et al. [4].

The lawyer recommendation engine produces ranked, explainable results across over 20 legal specialisation categories. The scoring formula - specialisation match (+20), biography term frequency (+2/term), availability bonus (+10) - is interpretable, auditable, and easily extensible to additional scoring dimensions such as geographic proximity or rating percentile. The forum's PageRank-style contributor ranking and DFS thread reconstruction are fully implemented and functional in the live application, demonstrating that graph algorithms from the theoretical literature can be productively applied in a real-time serverless database environment.

All eight system test cases pass, and all non-functional requirements - response latency, security, cost, scalability, and accessibility - are met. The platform is live at `https://nyayak.vercel.app` and deployed entirely on free-tier cloud infrastructure, confirming that production-quality access-to-justice technology does not require institutional hosting budgets.

Beyond its technical contributions, Nyayak represents the first openly documented full-stack legal AI platform designed specifically for Nepal's legal corpus. As such, it provides a practical reference architecture for applying modern information retrieval and graph algorithms to domain-specific, resource-constrained South Asian access-to-justice problems. The codebase is maintained on GitHub (github.com/paudelsamir/nyayak) with documentation sufficient for future extension by students or developers.

6.2. Future Recommendations

The current implementation provides a functional foundation that can be expanded as additional legal data, evaluation resources, and production infrastructure become available. The following recommendations prioritise accessibility, corpus coverage, verification, communication, evaluation, and long-term scalability.

Multilingual Support: Extend the query interface and retrieval pipeline to support native Nepali Devanagari script. This would require a multilingual embedding model fine-tuned on Nepali legal text, a transliteration preprocessing layer, and corpus re-ingestion in Unicode-normalised Devanagari.

Legal Corpus Expansion: Ingest provincial legislation, the Labour Act 2074, the Land Act 2021, criminal procedure statutes, and court procedural guidelines into ChromaDB. This would address the 14% of queries that currently return sub-optimal citations for procedural matters.

Nepal Bar Council API Integration: Replace the manual lawyer verification workflow with programmatic integration against the Nepal Bar Council's official registry, enabling real-time credential verification and automated specialisation tagging.

Mobile Application: Develop a React Native or Flutter mobile application to extend reach to rural communities with limited desktop access and lower smartphone storage constraints.

Full PageRank Convergence: Implement iterative PageRank convergence with a scheduled Convex background function rather than the current score approximation, enabling more accurate long-term contributor ranking as the forum scales.

Court Data Completeness: Supplement the court navigation graph with verified geodata for all sub-district and appellate courts, enabling routing beyond district and high court recommendations to specialised bench referrals.

AI Model Fine-Tuning: Fine-tune a smaller open-source LLM (e.g., Llama 3 8B or Mistral 7B) on Nepali legal question-answer pairs to reduce dependence on third-party inference APIs and improve accuracy on Nepal-specific legal nuances.

Lawyer Booking Payment Integration: Integrate an Esewa or Khalti payment gateway for consultation booking fees, enabling monetisation of the lawyer marketplace while keeping AI legal guidance free for citizens.

Real-Time Communication Support: Add in-platform real-time chat with end-to-end encrypted messaging and secure audio/video calling support between citizens and lawyers, enabling confidential consultation without relying on third-party communication tools.

Evaluation Expansion: Extend the RAG benchmark from 50 to 500 queries, covering the full range of constitutional articles, civil code sections, and procedural statutes, to provide a statistically robust evaluation of retrieval accuracy across the entire legal corpus.

References

[1] Nepal Law Commission, "Constitution of nepal 2015 (2072 b.s.)," Kathmandu, Nepal, 2015.

[2] Government of Nepal, "Civil code 2017," Kathmandu, Nepal, 2017.

[3] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[4] Z. Shao, Y. Gong, Y. Shen *et al.*, "Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy," arXiv, 2023. [Online]. Available: https://arxiv.org/abs/2305.15294

[5] S. Abirami, M. J. Joshma, A. Jayavarthini, A. Joshi, and M. K. Gajendran, "Evaluating lexical, dense, and hybrid retrieval pipelines for rag," IEEE Xplore, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11379254/

[6] R. Burke, "Knowledge-based recommender systems," 2000. [Online]. Available: http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.21.6029

[7] G. Adomavičius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, 2005. [Online]. Available: http://ieeexplore.ieee.org/document/1423975/

[8] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep., 1999. [Online]. Available: http://ilpubs.stanford.edu:8090/422/

[9] X. Z. Wang, "The comparison of three algorithms in shortest path issue," *Journal of Physics Conference Series*, vol. 1087, pp. 022 011–022 011, 2018. [Online]. Available: https://iopscience.iop.org/article/10.1088/1742-6596/1087/2/022011

[10] F. Ariai, J. Mackenzie, and G. Demartini, "Natural language processing for the legal domain: A survey of tasks, datasets, models, and challenges," *ACM Computing Surveys*, vol. 58, no. 6, pp. 1–37, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3777009

[11] H. A. Jartarghar, G. R. Salanke, A. K. A. R., S. G.S, and S. Dalali, "React apps with server-side rendering: Next.js," *Journal of Telecommunication Electronic and*

Computer Engineering (JTEC), vol. 14, no. 4, pp. 25–29, 2022. [Online]. Available: https://jtec.utem.edu.my/jtec/article/view/6192

[12] D. A. Laila, Q. Al-Na'amneh, M. Aljawarneh, W. Dhifallah, and K. S. Maabreh, "Retrieval-augmented generation with large language models," 2025. [Online]. Available: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-5017-2.ch003

[13] M. Dahl, V. Magesh, M. Suzgun, and D. E. Ho, "Large legal fictions: Profiling legal hallucinations in large language models," *The Journal of Legal Analysis*, vol. 16, no. 1, pp. 64–93, 2024. [Online]. Available: https://academic.oup.com/jla/article/16/1/64/7699227

[14] S. Gupta, V. Agrawal, Y. Negi, S. Karunanithi, and A. Balakrishnan, "Reducing hallucinations in legal ai: A retrieval augmented generation-based model for accurate legal guidance," *IEEE Access*, vol. 14, pp. 44 451–44 463, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11443248/

[15] R. E. Hamdani, T. Bonald, F. D. Malliaros, N. Holzenberger, and F. M. Suchanek, "The factuality of large language models in the legal domain," 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3627673.3679961

[16] n.n., "The pagerank citation ranking: Bringing order to the web," CiteSeerX Archive Copy, Tech. Rep., 1998. [Online]. Available: http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.102.9606

[17] T. Robertson, S. Walker, S. Jones, M. Hancock-Beaulieu, and M. Gatford, "Okapi at trec-3," in *NIST Special Publication 500-225*, 1995.

[18] G. M. Rosa, R. C. Rodrigues, R. Lotufo, and R. Nogueira, "Yes, bm25 is a strong baseline for legal case retrieval," arXiv, 2021. [Online]. Available: https://arxiv.org/abs/2105.05686

[19] M. Kim, J. Rabelo, K. C. Okeke, and R. Goebel, "Legal information retrieval and entailment based on bm25, transformer and semantic thesaurus methods," *The Review of Socionetwork Strategies*, vol. 16, no. 1, pp. 157–174, 2022. [Online]. Available: https://link.springer.com/10.1007/s12626-022-00103-1

[20] C. S. Mala, G. Gezici, and F. Giannotti, "Hybrid retrieval for hallucination mitigation in large language models: A comparative analysis," arXiv, 2025. [Online]. Available: https://arxiv.org/abs/2504.05324

[21] H. Zhong, C. Xiao, C. Tu, T. Zhang, Z. Liu, and M. Sun, "How does nlp benefit legal system: A summary of legal artificial intelligence," in *Proceedings of the 58th Annual*

Meeting of the Association for Computational Linguistics, 2020. [Online]. Available: https://www.aclweb.org/anthology/2020.acl-main.466

[22] A. Dantart, "Reliability by design: Quantifying and eliminating fabrication risk in llms. from generative to consultative ai: A comparative analysis in the legal domain and lessons for high-stakes knowledge bases," arXiv, 2026. [Online]. Available: https://arxiv.org/abs/2601.15476

[23] P. Zhao, H. Zhang, Q. Yu, Z. Wang, Y. Geng, F. Fu, L. Yang, W. Zhang, J. Jiang, and B. Cui, "Retrieval-augmented generation for ai-generated content: A survey," arXiv, 2024. [Online]. Available: https://arxiv.org/abs/2402.19473

[24] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of EMNLP*, 2019. [Online]. Available: https://www.aclweb.org/anthology/D19-1410

[25] Nepal Telecommunications Authority, "Management information system (mis) report 2023," Kathmandu, Nepal, 2023.

A. Screenshots of the System

The following screenshots document the deployed Nyayak user-facing modules available at `https://nyayak.vercel.app`.

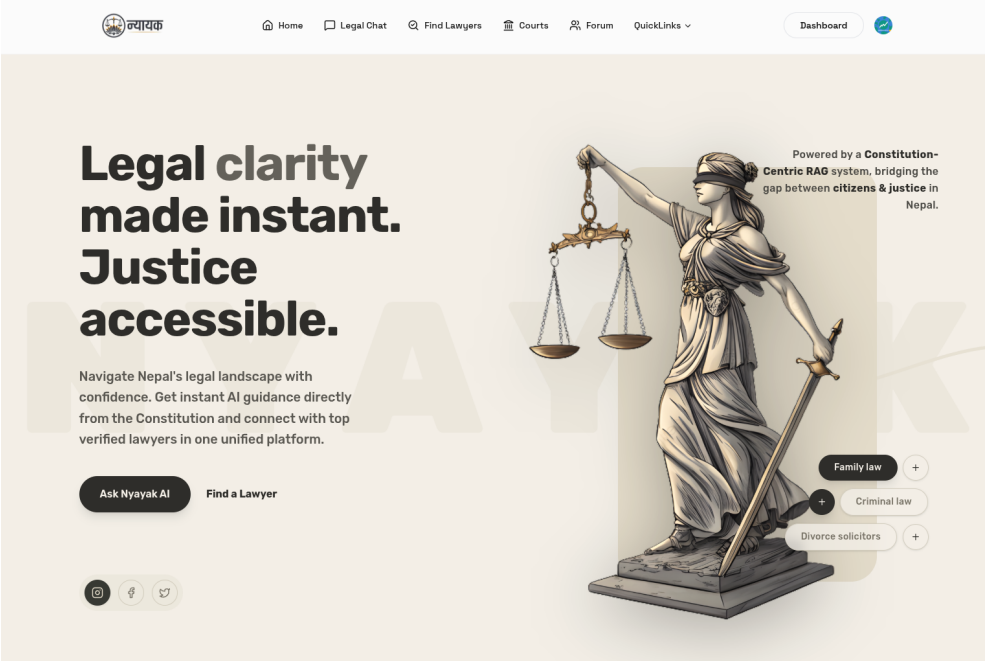

Figure A.1: Landing Page - Desktop

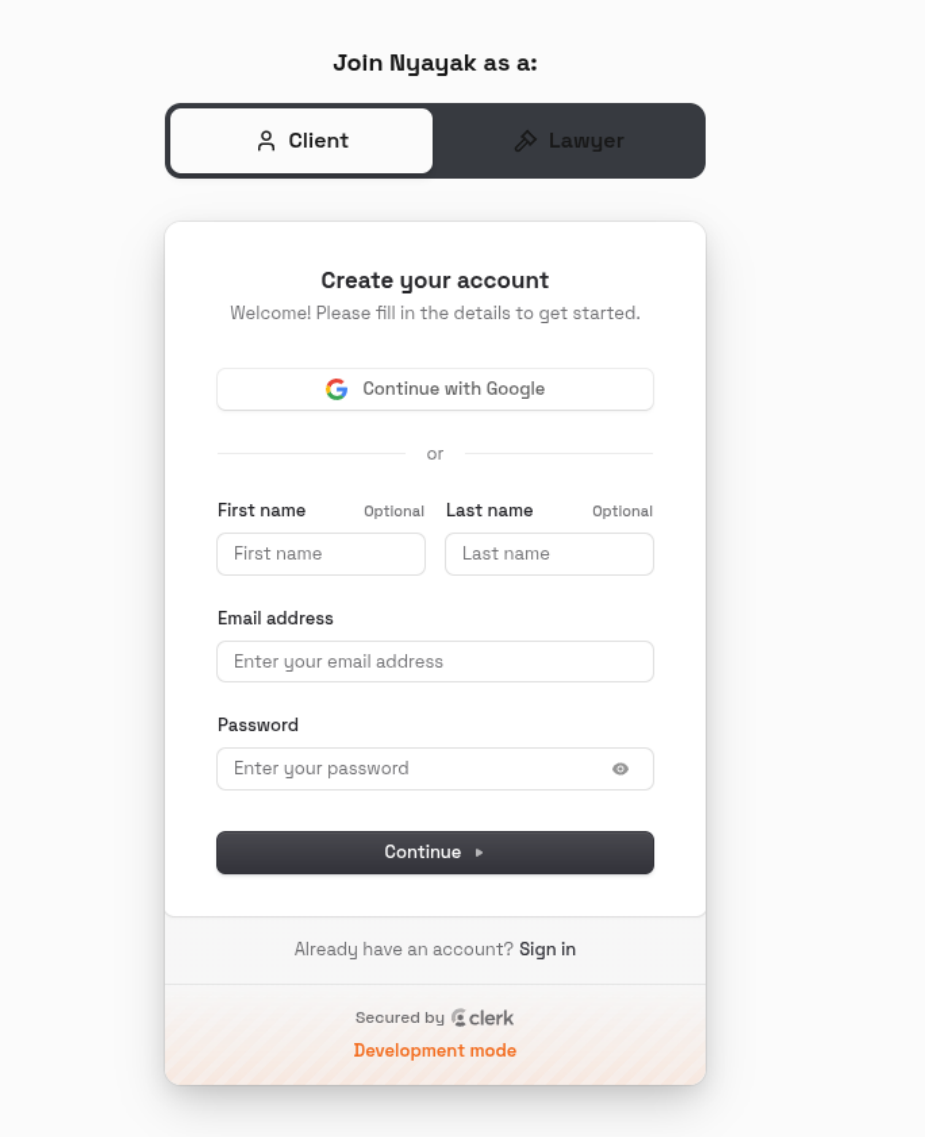

Figure A.2: Clerk Authentication Signup Page

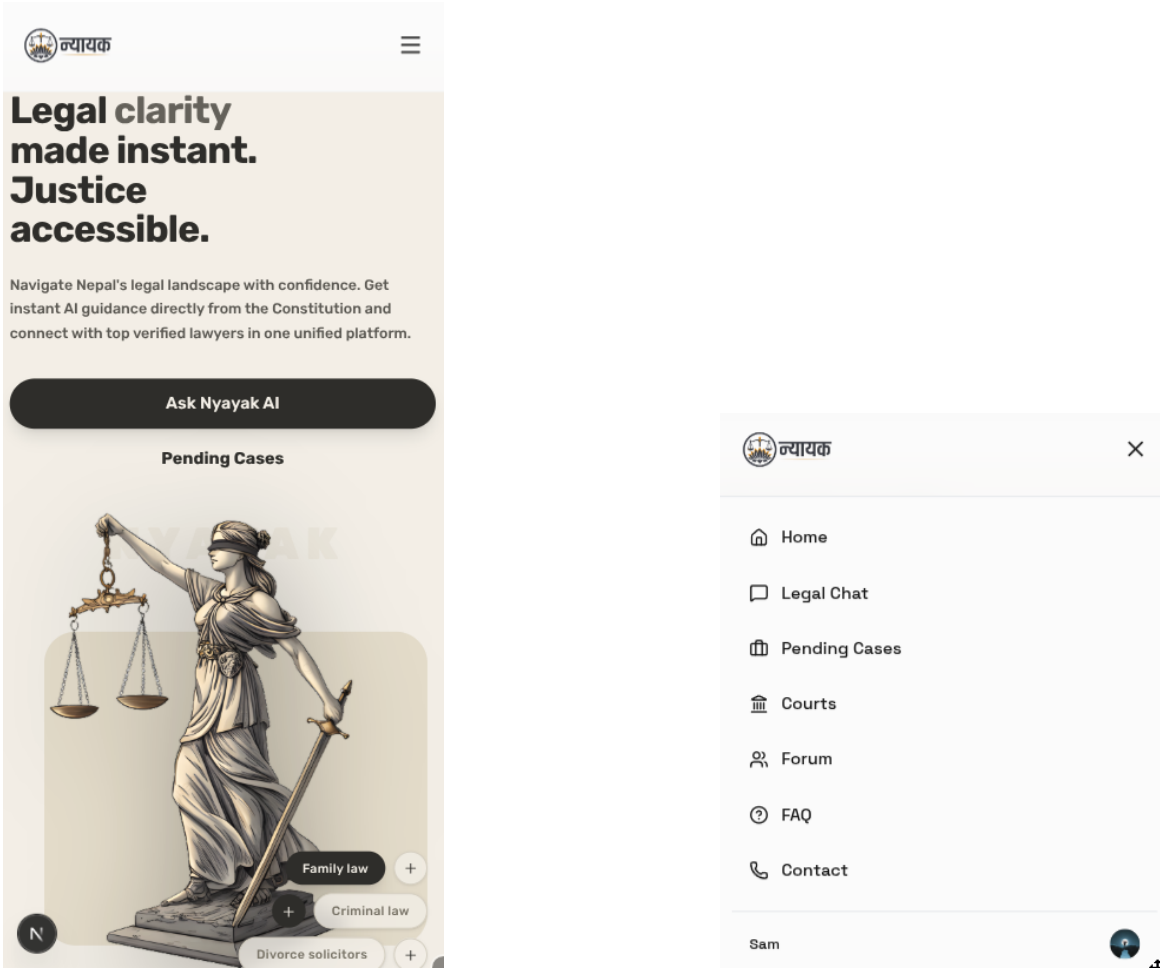

Figure A.3: Landing Page - Mobile and Navigation Drawer

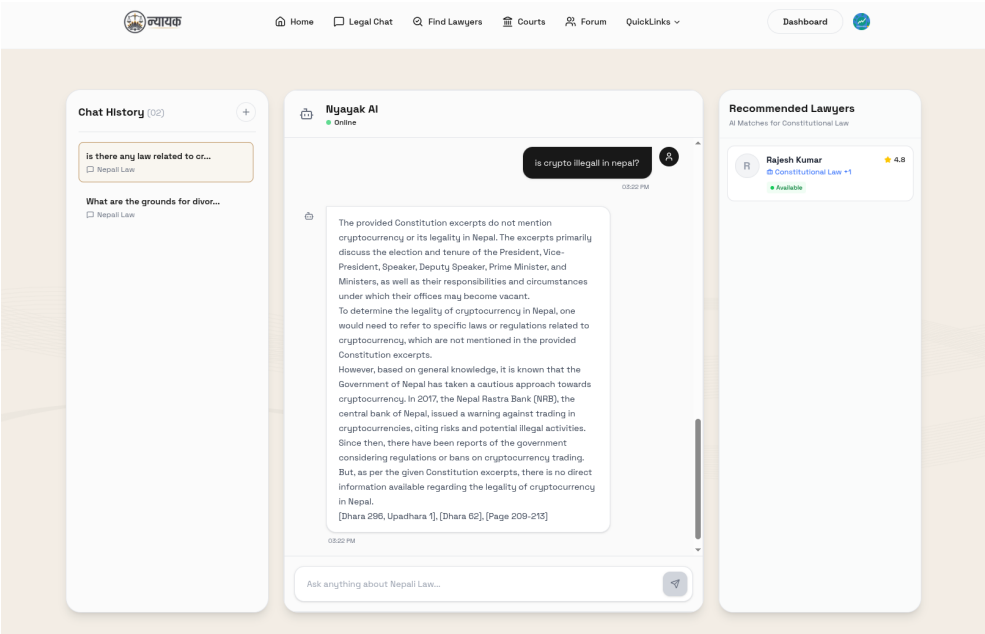

Figure A.4: AI Chat Interface with Lawyer Recommendation Panel

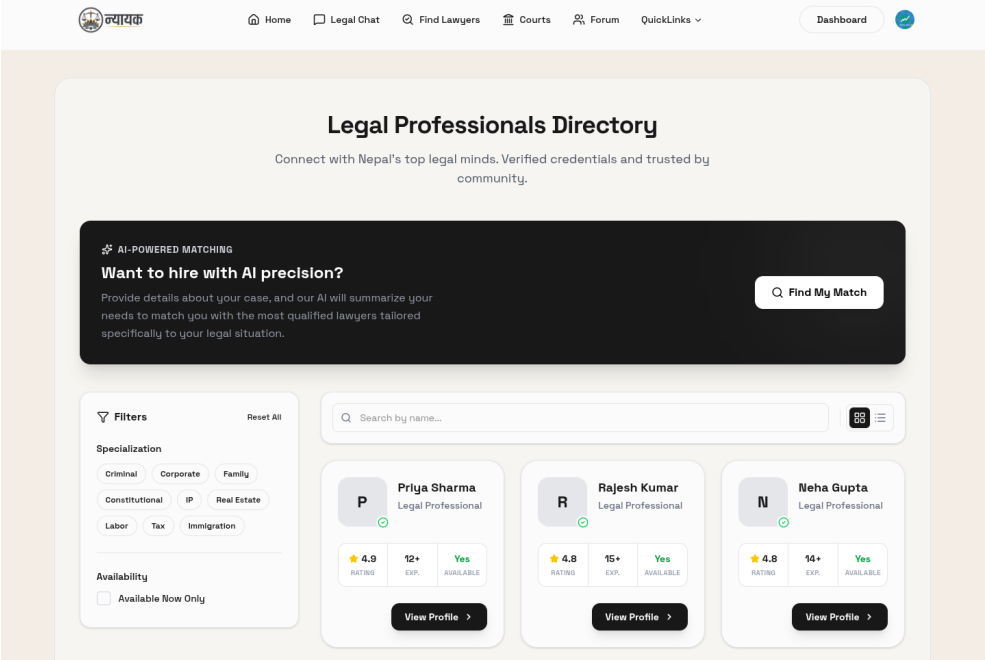

Figure A.5: Lawyer Directory - Find Lawyer Section

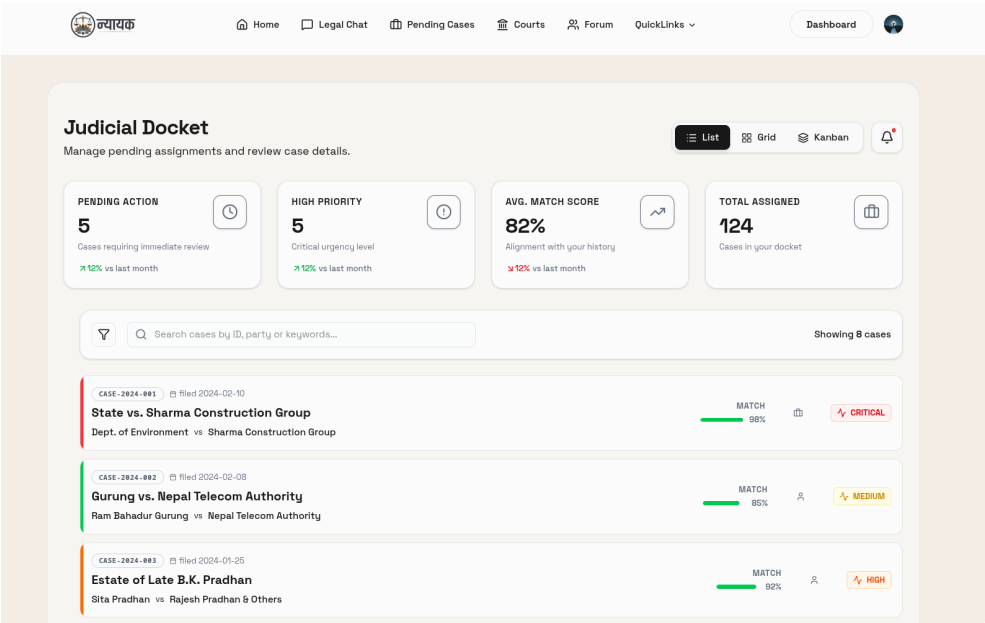

Figure A.6: Pending Cases - Lawyer Docket View

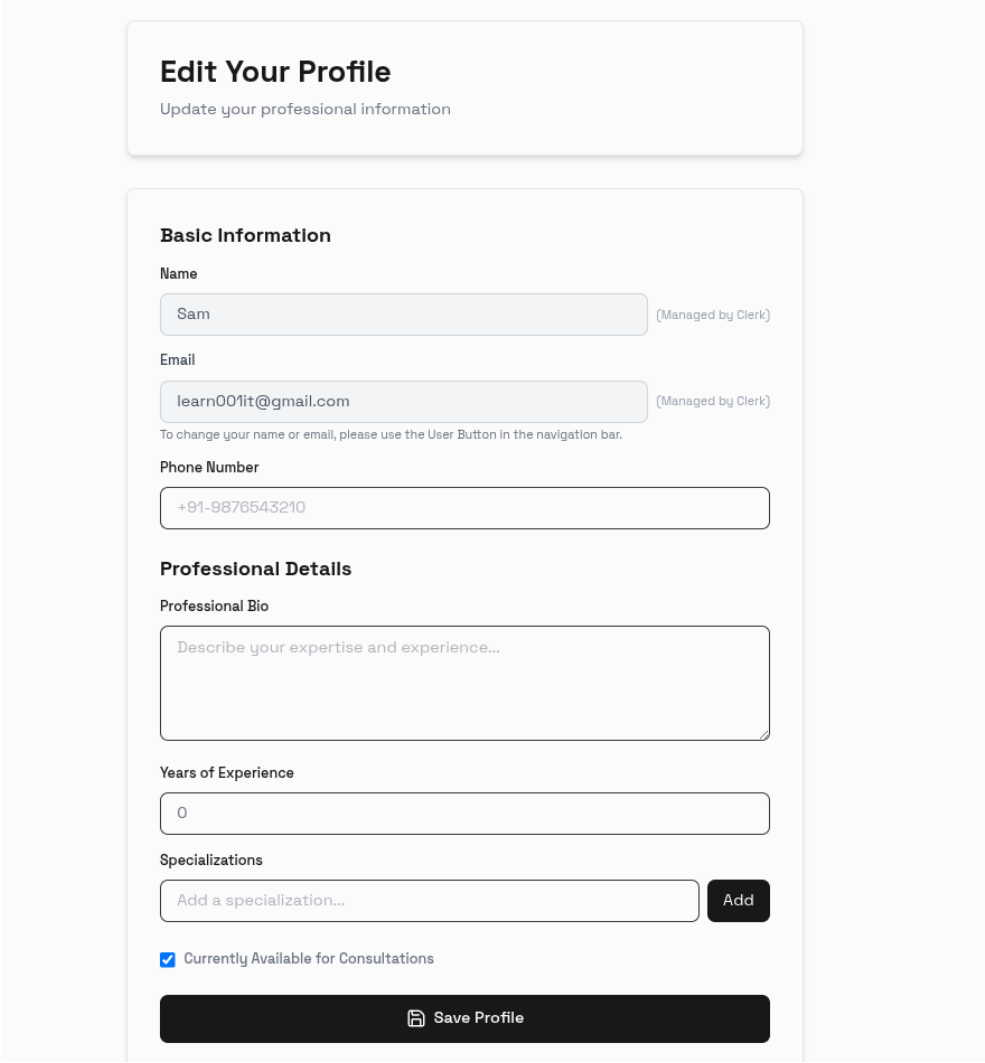

Figure A.7: Lawyer Edit Profile Section

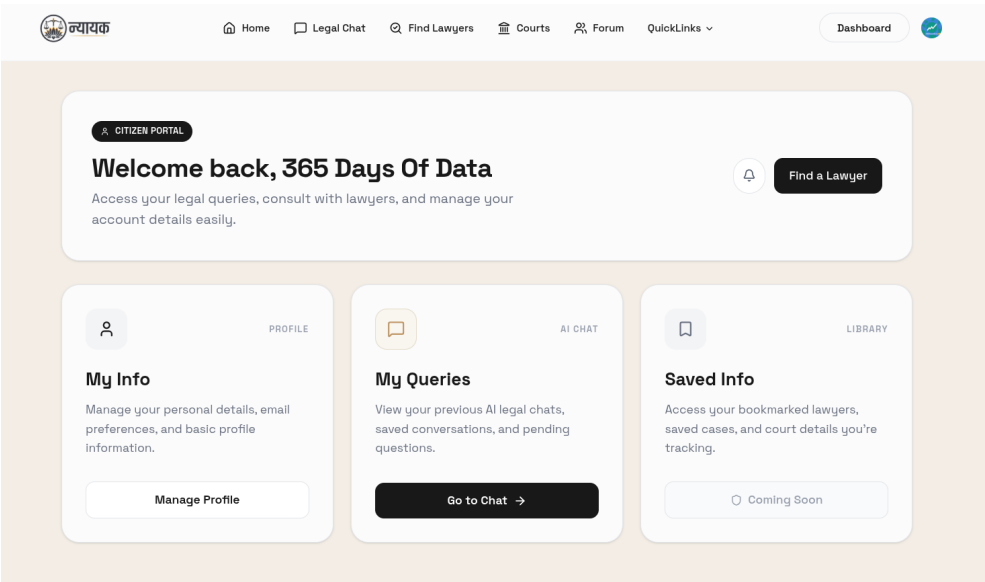

Figure A.8: Citizen Dashboard

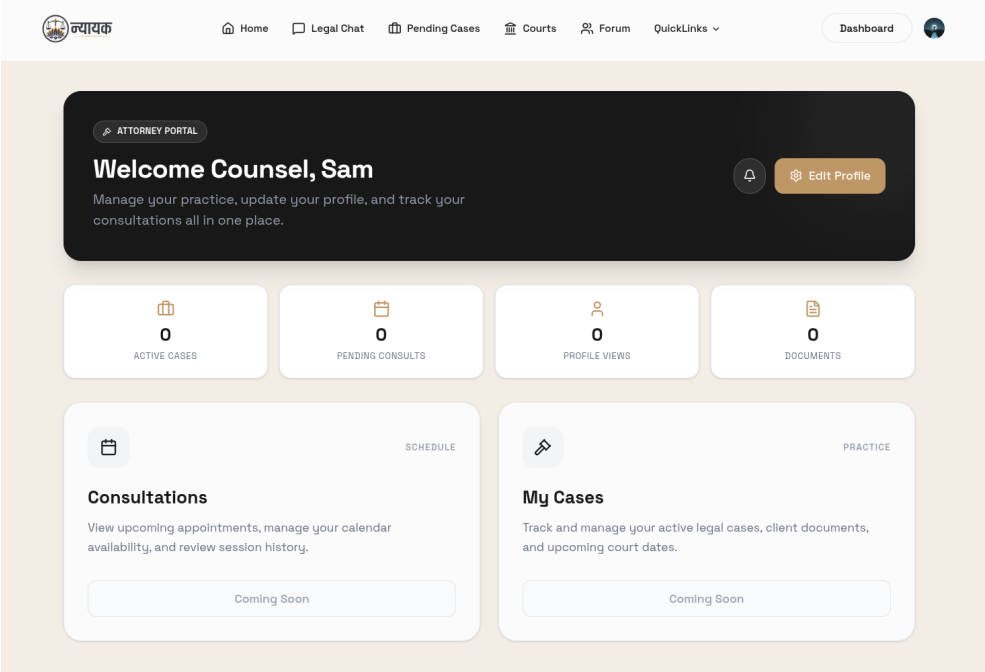

Figure A.9: Lawyer Dashboard

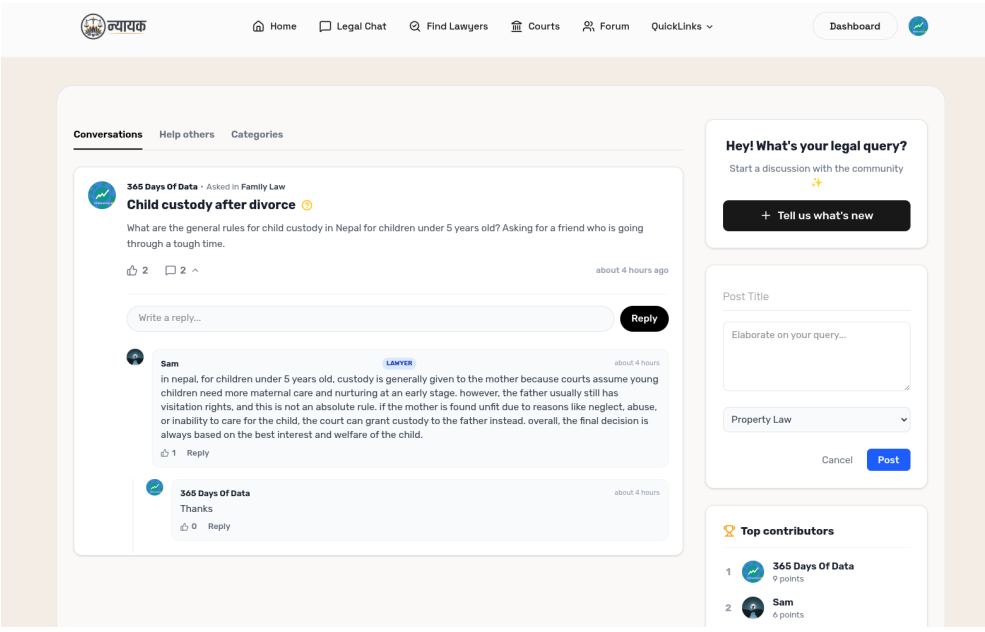

Figure A.10: Community Forum - Nested Threads

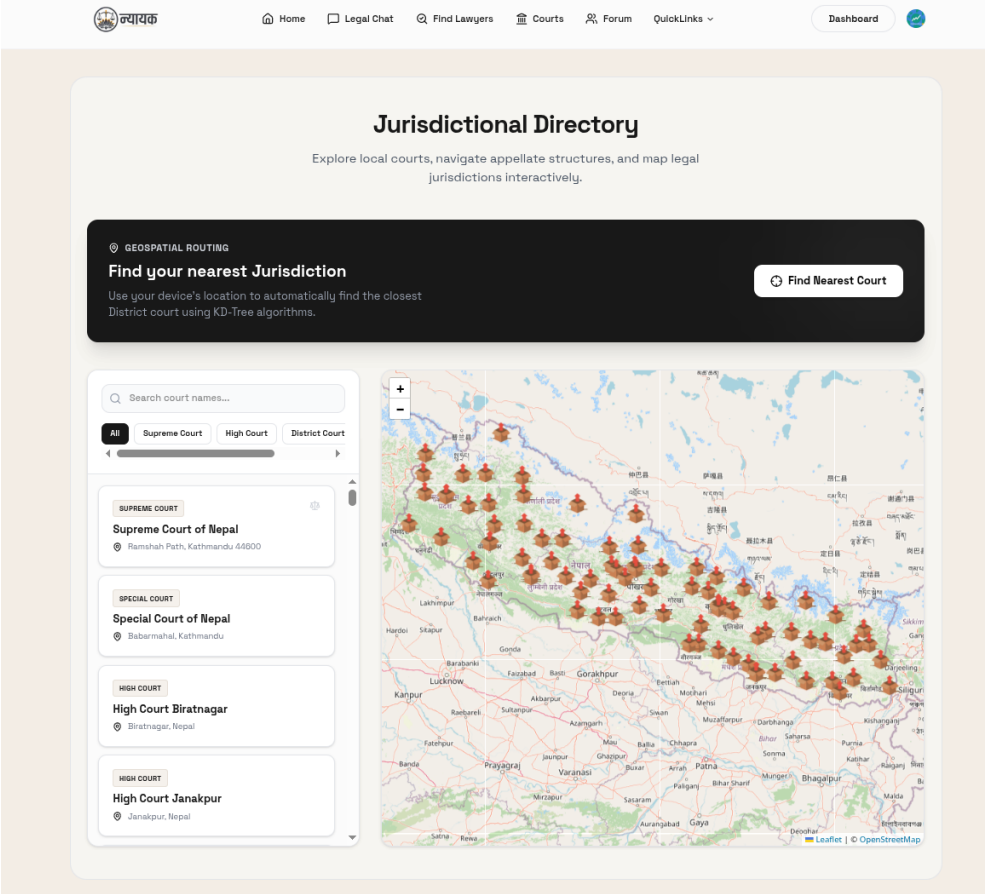

Figure A.11: Court Navigation Map

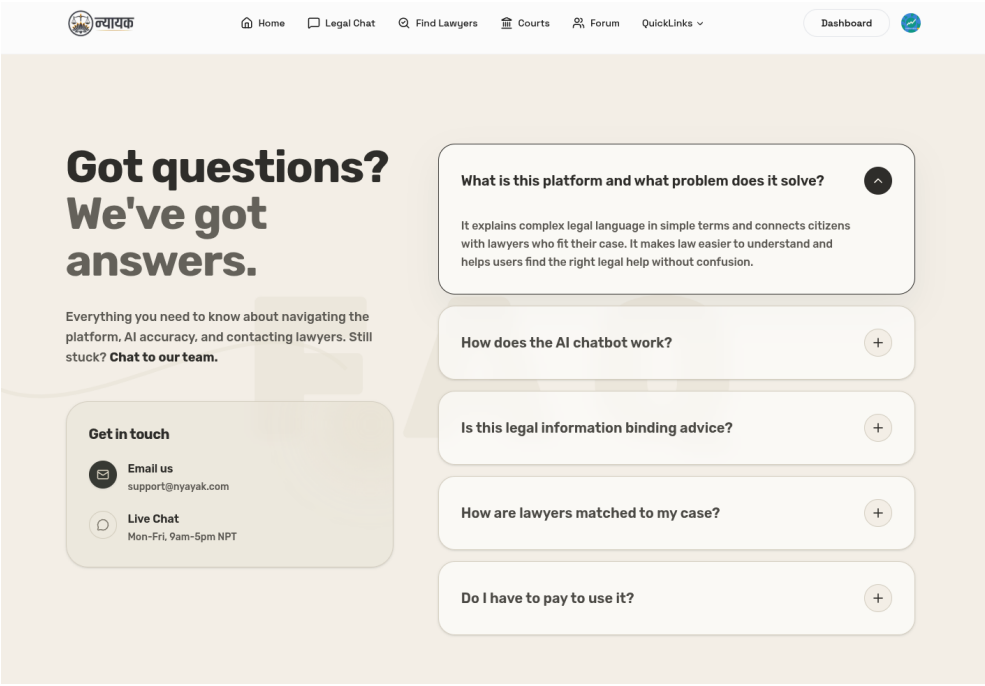

Figure A.12: FAQ Section

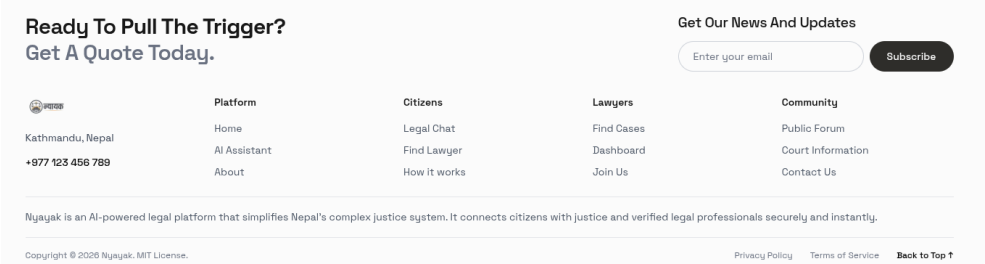

Figure A.13: Footer Section

B. Major Source Code Components

B.1 FastAPI Backend Entry Point (backend/main.py)

```python
import os
import logging
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from dotenv import load_dotenv
from groq import Groq
from models import LegalQuery, LegalResponse
from rag_pipeline import RAGPipeline, load_constitution_pdf

# Initialize FastAPI app
app = FastAPI(
    title="Synergy Legal Assistant API",
    description="Python backend for RAG-powered legal consultation",
    version="1.0.0"
)

# Enable CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "http://localhost:3001"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Initialize RAG pipeline
constitution_docs = load_constitution_pdf()
rag_pipeline = RAGPipeline(constitution_docs)

```

```python
@app.post("/api/ask-legal", response_model=LegalResponse)
async def ask_legal(request: LegalQuery):
    """Legal consultation endpoint with RAG"""
    query = request.query
    relevant_docs = rag_pipeline.query(query)
    # Process and return response
    return {"answer": answer, "sources": sources}
```

B.2 RAG Pipeline (backend/rag_pipeline.py)

```python
class HybridVectorStore:
    """Fast hybrid search: BM25 keyword + semantic embeddings"""

    def hybrid_search(self, query: str, k: int = 3) -> List[Dict]:
        """Hybrid search combining BM25 and semantic similarity"""
        results = {}

        # BM25 keyword search (70% weight)
        if self.bm25:
            query_tokens = query.lower().split()
            bm25_scores = self.bm25.get_scores(query_tokens)
            for idx, score in enumerate(bm25_scores):
                if score > 0.01:
                    results[idx] = score * 0.7

        # Semantic search with embeddings (30% weight)
        if self.embeddings and self.doc_embeddings is not None:
            query_embedding = self.embeddings.encode(query)
            similarities = cosine_similarity(
                [query_embedding], self.doc_embeddings)[0]
            for idx, score in enumerate(similarities):
                if score > 0.1:
                    results[idx] = results.get(idx, 0) + (score * 0.3)

        sorted_results = sorted(results.items(),
                                key=lambda x: x[1], reverse=True)[:k]
        return [self.documents[idx] for idx, _ in sorted_results]
```

B.3 Convex Database Schema (convex/schema.ts)

```typescript
export default defineSchema({
  users: defineTable({
    name: v.optional(v.string()),
    email: v.string(),
    clerkId: v.string(),
    role: v.union(v.literal("user"),
                  v.literal("lawyer"),
```

```
                  v.literal("admin"))
  }).index("by_clerkId", ["clerkId"]),

  lawyers: defineTable({
    userId: v.id("users"),
    specializations: v.array(v.string()),
    yearsOfExperience: v.number(),
    rating: v.number(),
    phone: v.string(),
    bio: v.string(),
    availableNow: v.boolean()
  }).index("by_userId", ["userId"]),

  forum_posts: defineTable({
    title: v.string(),
    content: v.string(),
    authorId: v.id("users"),
    category: v.optional(v.string()),
    upvotes: v.number()
  }).index("by_authorId", ["authorId"]),

  forum_comments: defineTable({
    postId: v.id("forum_posts"),
    authorId: v.id("users"),
    content: v.string(),
    parentId: v.optional(v.id("forum_comments")),
    upvotes: v.number()
  }).index("by_postId", ["postId"])
});
```

B.4 Lawyer Recommendation Logic (convex/lawyers.ts)

```
export const recommend = query({
  args: { query: v.string() },
  handler: async (ctx, args) => {
    const lawyerProfiles = await ctx.db.query("lawyers").collect();
    const keywords = extractKeywords(args.query);

    const lawyers = await Promise.all(
      lawyerProfiles.map(async (profile) => {
        let score = 0;

        // 1. Specialization Match (20 points)
        const exactMatch = profile.specializations.some(s =>
          args.query.toLowerCase().includes(s.toLowerCase())
        );
        if (exactMatch) score += 20;
```

```
        // 2. Keyword Match in Bio (2 points per hit)
        const bioLower = profile.bio.toLowerCase();
        keywords.forEach(k => {
          if (bioLower.includes(k)) score += 2;
        });

        // 3. Rating Bonus (1-5 points)
        score += (profile.rating || 0);

        // 4. Experience Bonus (up to 5 points)
        score += Math.min(profile.yearsOfExperience * 0.2, 5);

        // 5. Availability Bonus (3 points)
        if (profile.availableNow) score += 3;

        return { ...profile, score, user };
      })
    );
    return lawyers.sort((a, b) => b.score - a.score);
  }
});
```

B.5 Forum PageRank and DFS (`convex/forum.ts`)

```
export const getCommentsAndTraverse = query({
  args: { postId: v.id("forum_posts") },
  handler: async (ctx, args) => {
    const comments = await ctx.db.query("forum_comments")
      .withIndex("by_postId", q => q.eq("postId", args.postId))
      .collect();

    // Build adjacency list for threading
    const graph: Record<string, any[]> = {};
    const roots: any[] = [];

    comments.forEach((comment) => { graph[comment._id] = []; });

    comments.forEach((comment) => {
      if (comment.parentId && graph[comment.parentId]) {
        graph[comment.parentId].push(comment);
      } else {
        roots.push(comment);
      }
    });

    // DFS Traversal for chronological thread reconstruction
```

```
    const flattenTreeDFS = (nodeList: any[], depth = 0): any[] => {
      let result: any[] = [];
      for (const node of nodeList) {
        result.push({ ...node, depth });
        const children = graph[node._id] || [];
        if (children.length > 0) {
          result = result.concat(flattenTreeDFS(children, depth + 1));
        }
      }
      return result;
    };

    return flattenTreeDFS(roots);
  }
});
```

B.6 Chat Interface Component (src/components/chat/ChatInterface.tsx)

```
const sendMessage = async (text: string) => {
  if (!text.trim()) return;

  let currentSessionId = activeSessionId;

  // Create session if needed
  if (!currentSessionId) {
    const title = text.length > 30 ? text.substring(0, 30) : text;
    currentSessionId = await createSession({ title });
    onSessionCreated(currentSessionId);
  }

  // Save user message
  await addMessage({
    sessionId: currentSessionId,
    text,
    sender: "user",
  });

  setLoading(true);
  try {
    // Call backend RAG API
    const response = await fetch("/api/ask-legal", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ query: text }),
    });

    const data = await response.json();
```

```
30
31      // Save assistant response
32      await addMessage({
33        sessionId: currentSessionId,
34        text: data.answer,
35        sender: "assistant",
36      });
37    } catch (error) {
38      console.error(error);
39    } finally {
40      setLoading(false);
41    }
42  };
```

C. Project Progress Log Report